



■ CS177 Bioinformatics: Software Deveopment and Applications

Lecture 9: Natural Language Processing (NLP)

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2024



Learning objectives



- After taking this lecture, you should be able to:
 - Explain the motivation and basic ideas of attention mechanisms in the context of machine translation
 - Describe the overall architecture of Transformer
 - Explain a few key innovations of Transformer: self-attention, multi-head attention, and positional encoding
 - Describe the basic architectures of BERT and GPT



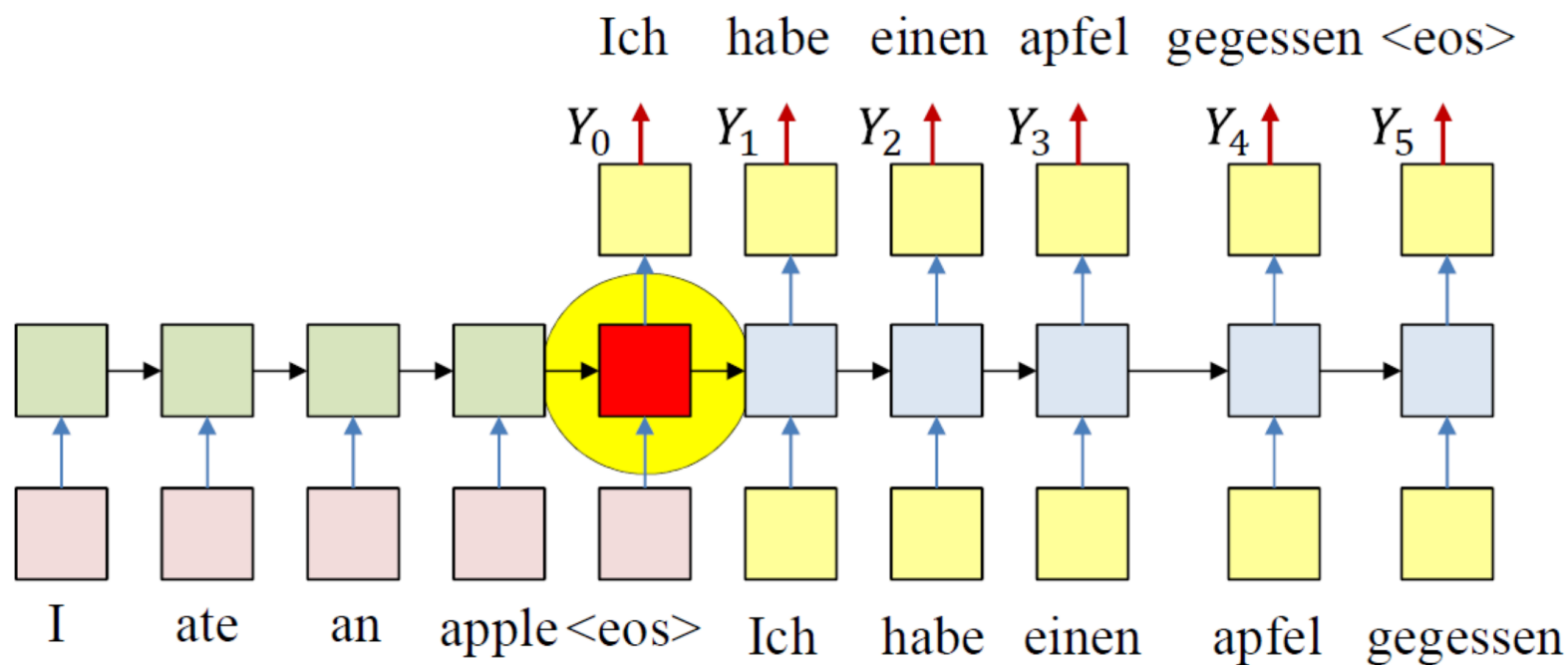


Attention Models



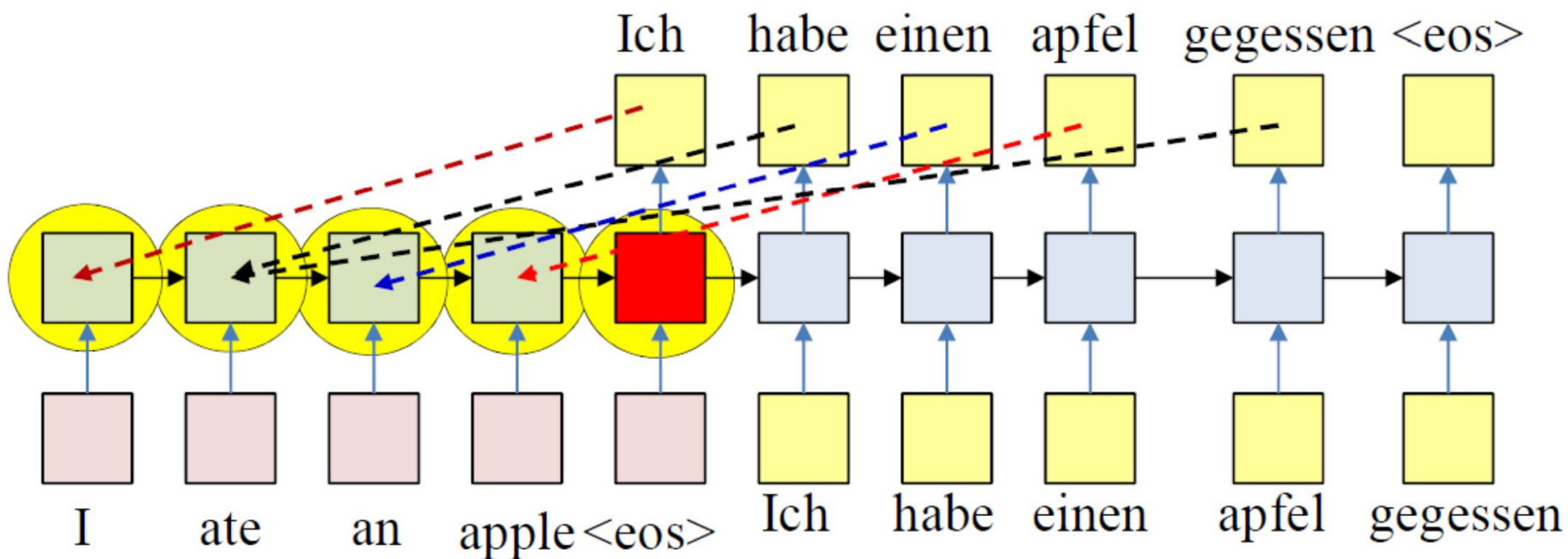
Recap: a problem with the encoder-decoder RNN framework

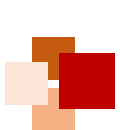
- All the information on the input sequence is embedded into a single vector
 - The latent layer at the end of the input sequence
 - This layer is overloaded with information



Recap: a problem with the encoder-decoder RNN framework

- All latent values carry information
 - Some of which may be diluted downstream
 - Different outputs are related to different inputs
 - Connecting everything to everything is infeasible
- This problem can be addressed using **Attention mechanisms**

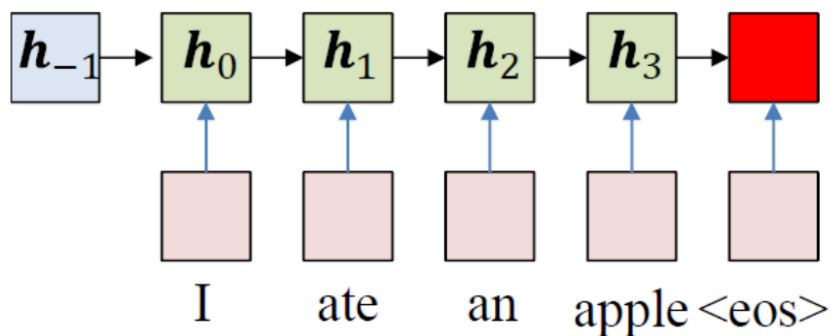
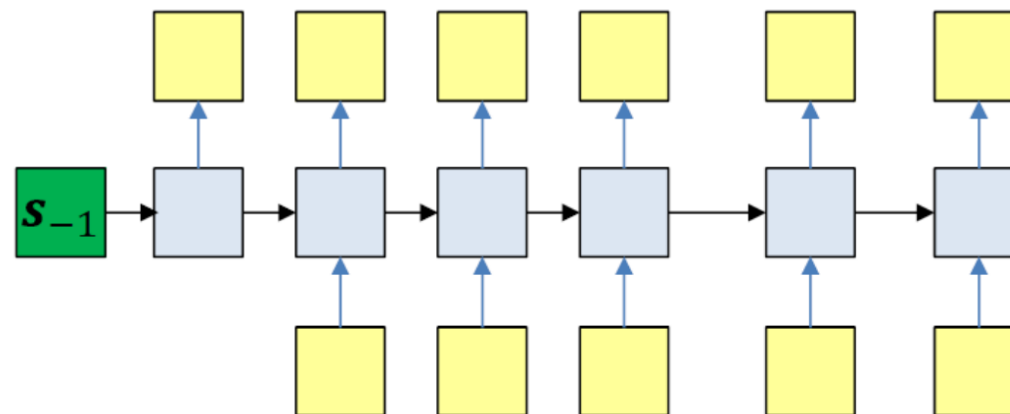


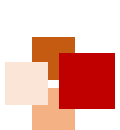


Attention models



- First, separate the encoder and decoder

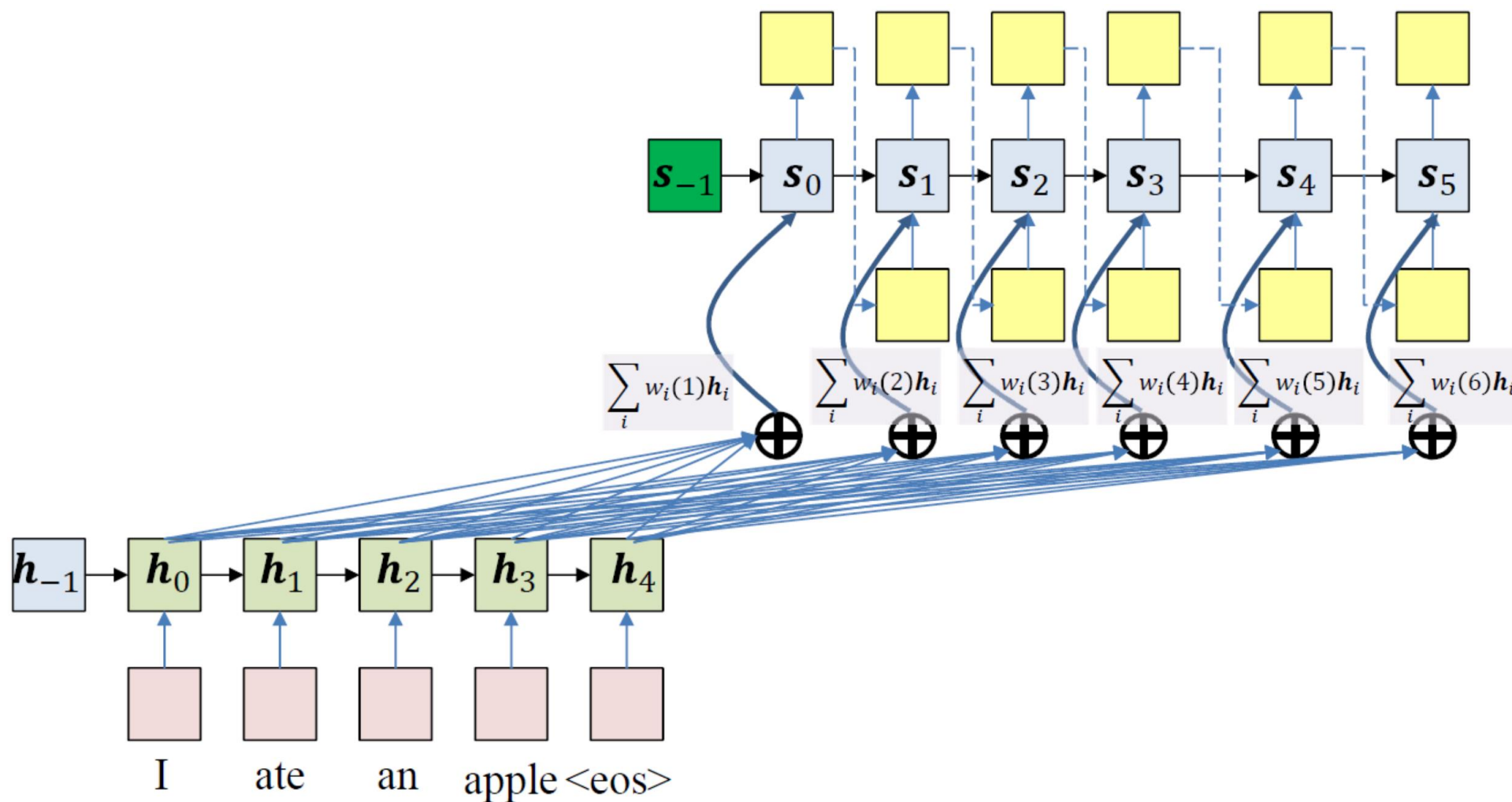




Attention models

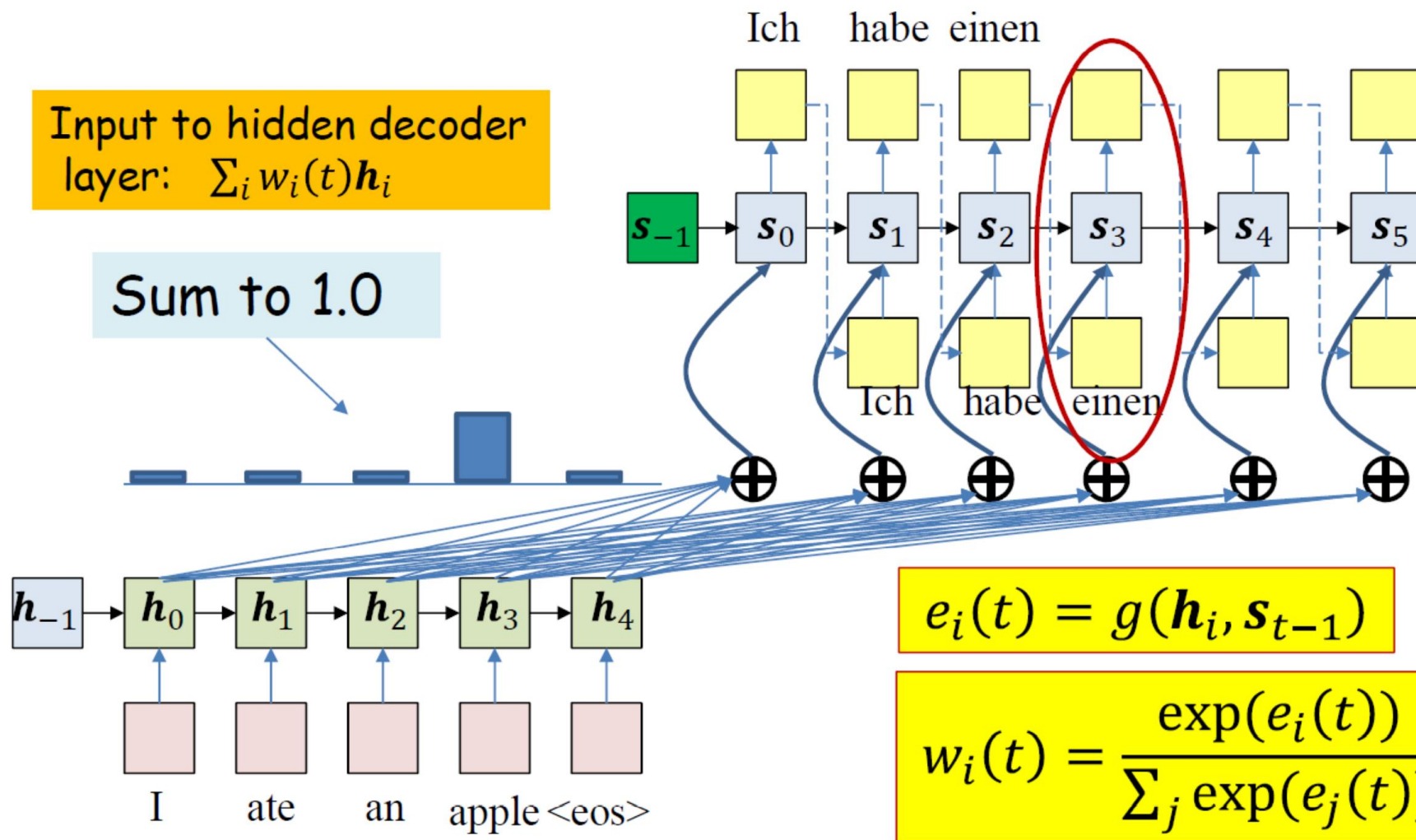


- Compute a weighted combination of all the hidden outputs into a single vector



Attention models

- The weights are a distribution over the input
 - A function $g()$ on two hidden states followed by a softmax



Attention models

- The weights are a distribution over the input
 - Typical options for $g()$ with **parameters** to be learned

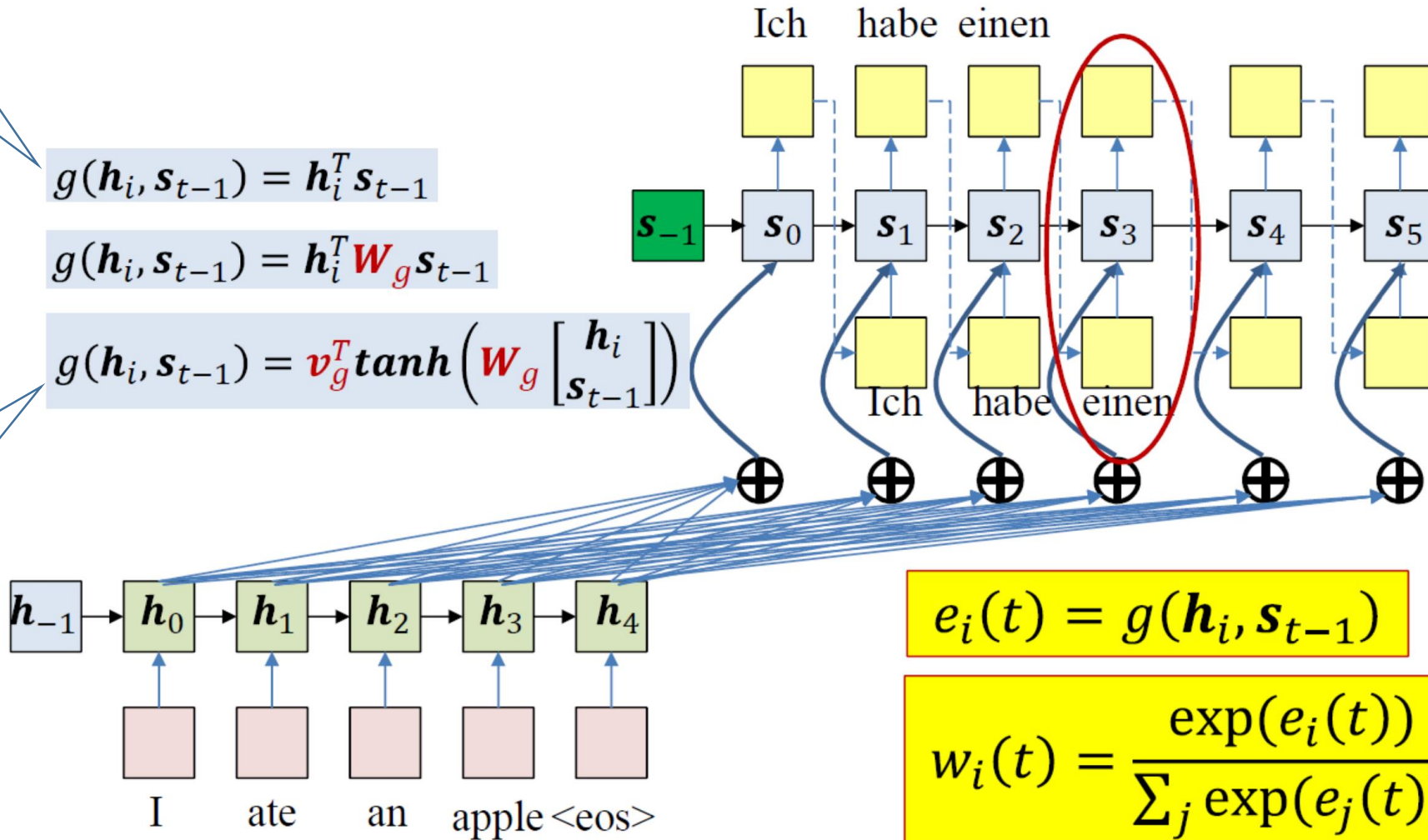
Luong
attention

$$g(h_i, s_{t-1}) = h_i^T s_{t-1}$$

$$g(h_i, s_{t-1}) = h_i^T W_g s_{t-1}$$

$$g(h_i, s_{t-1}) = v_g^T \tanh \left(W_g \begin{bmatrix} h_i \\ s_{t-1} \end{bmatrix} \right)$$

Bahdanau
attention

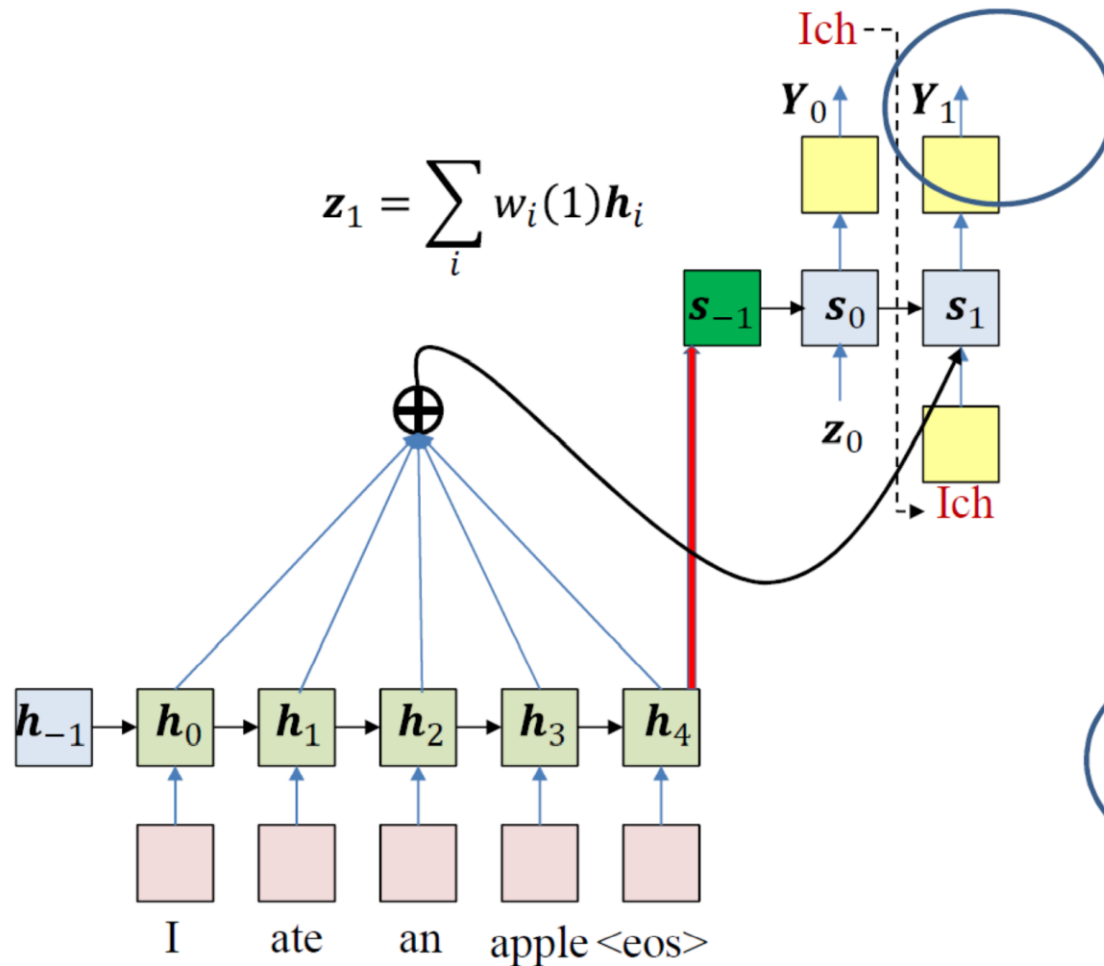


$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

What does the attention learn?

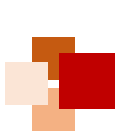
- The key component of this model is the attention weight
 - It captures the relative importance of each position in the input to the current output



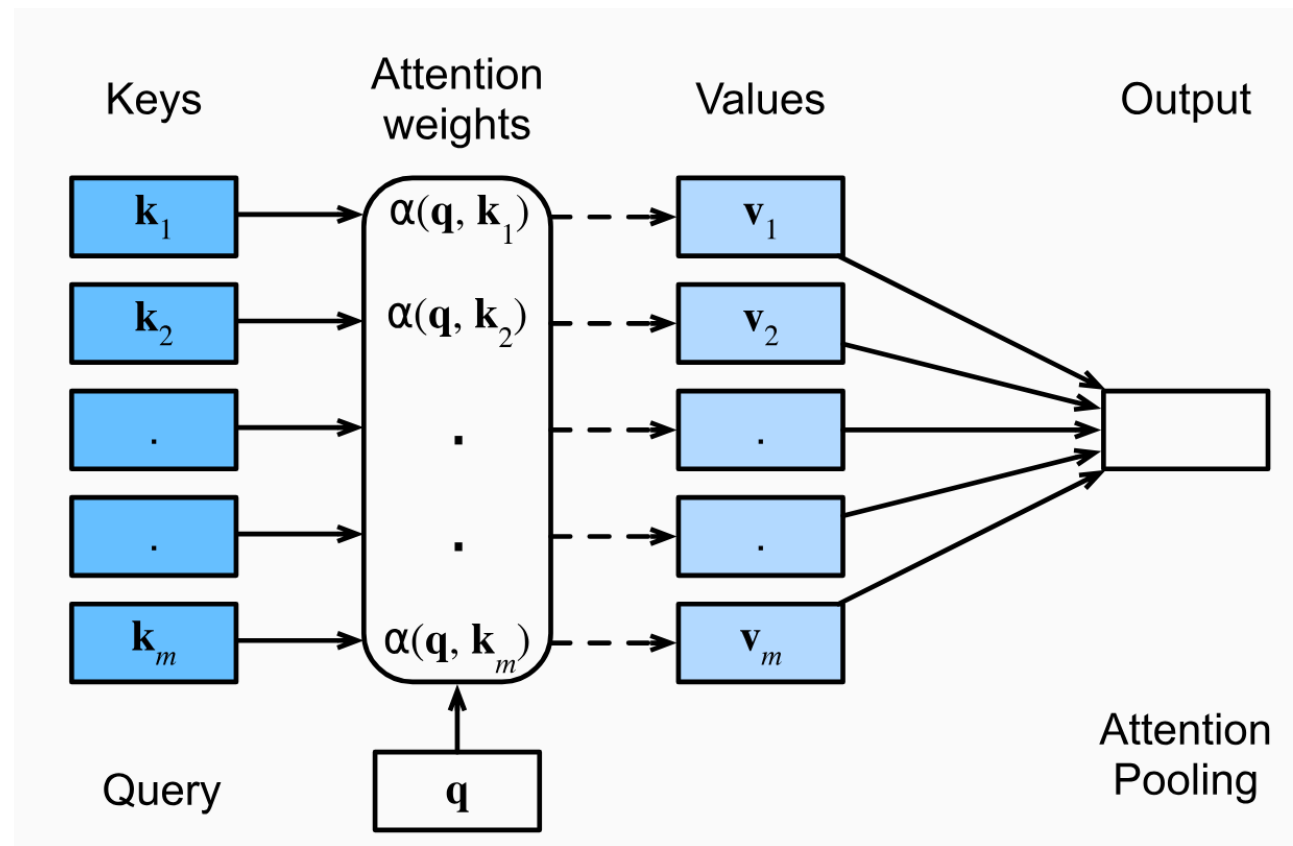
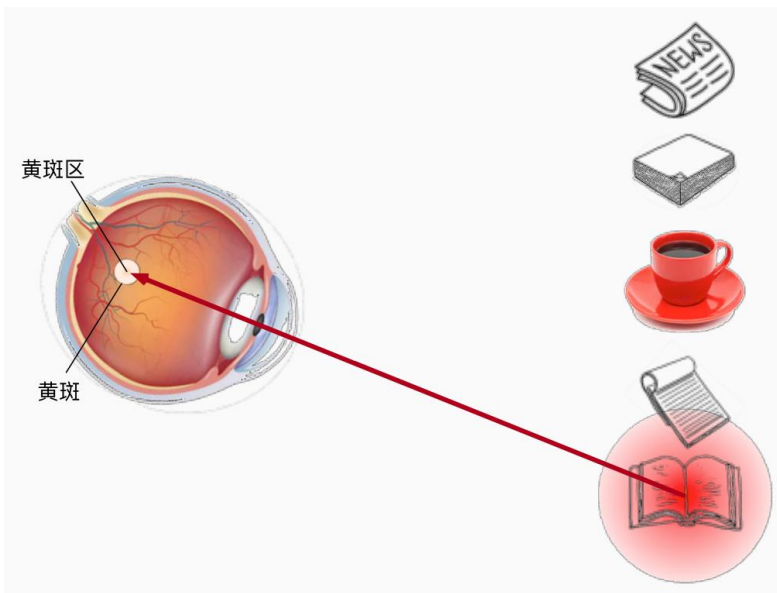
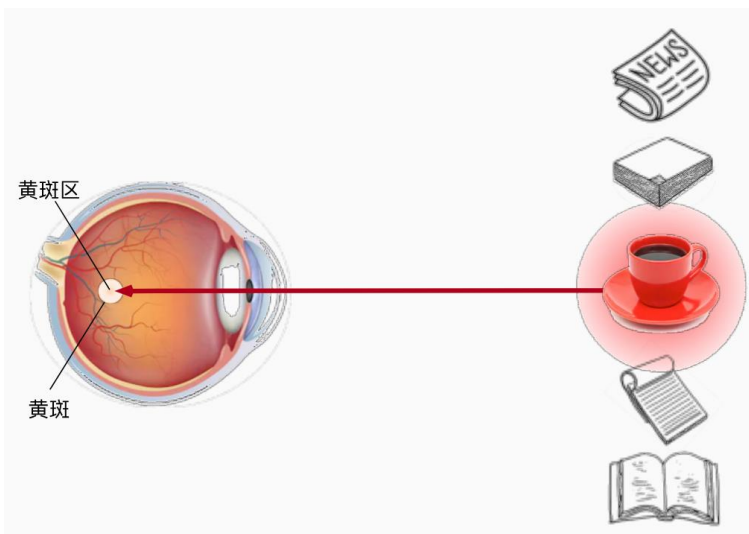
$$g(h_i, s_0) = h_i^T W_g s_0$$

$$e_i(1) = g(h_i, s_0)$$

$$w_i(1) = \frac{\exp(e_i(1))}{\sum_j \exp(e_j(1))}$$



Queries, Keys and Values



The attention mechanism computes a linear combination over values v_i via attention pooling, where weights are derived according to the compatibility between a query q and keys k_i .



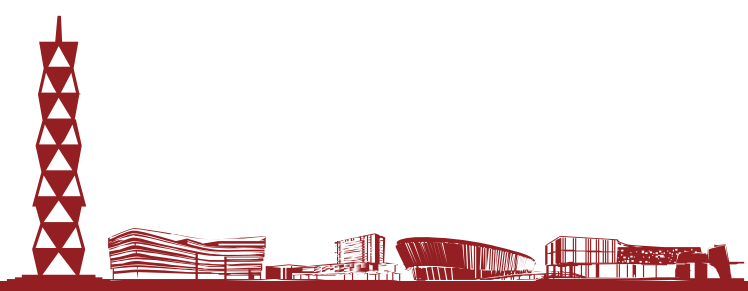
Transformer

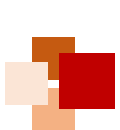


Limitations of RNNs



- RNNs involve sequential computation
 - Cannot parallelize => time-consuming
- RNNs “forget” past information
 - LSTM helps to some degree, but not too long
 - No explicit modeling of long and short-range dependencies

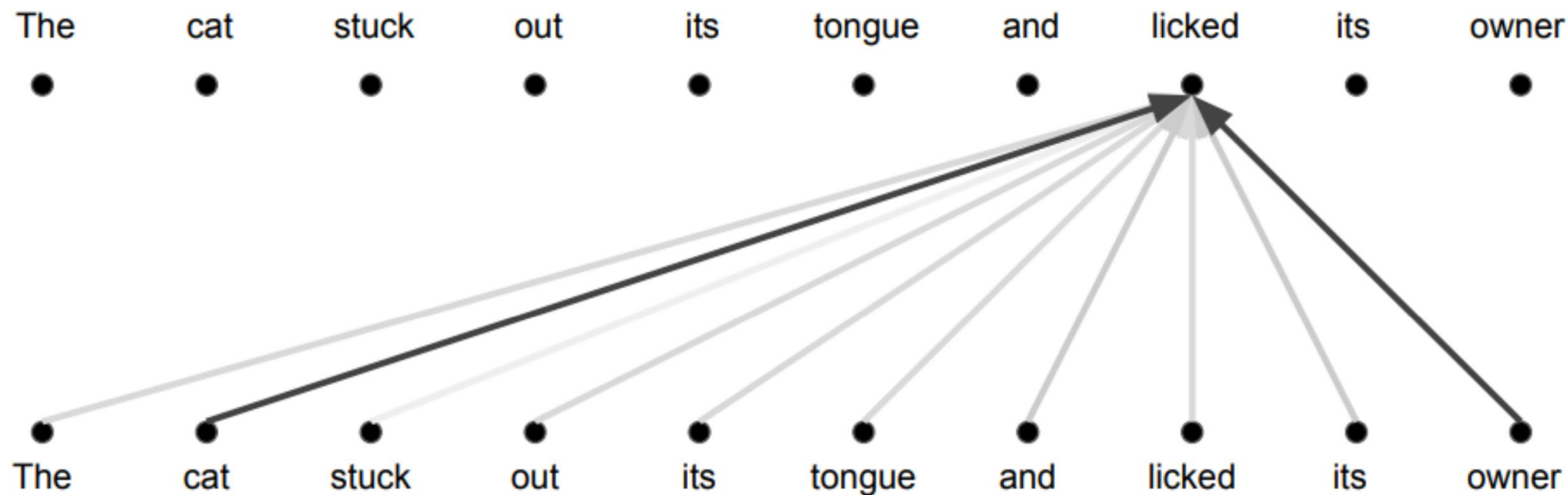




Attention

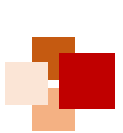


- Use (self-)attention as representation?



Ashish Vaswani & Anna Huang, Stanford CS224n

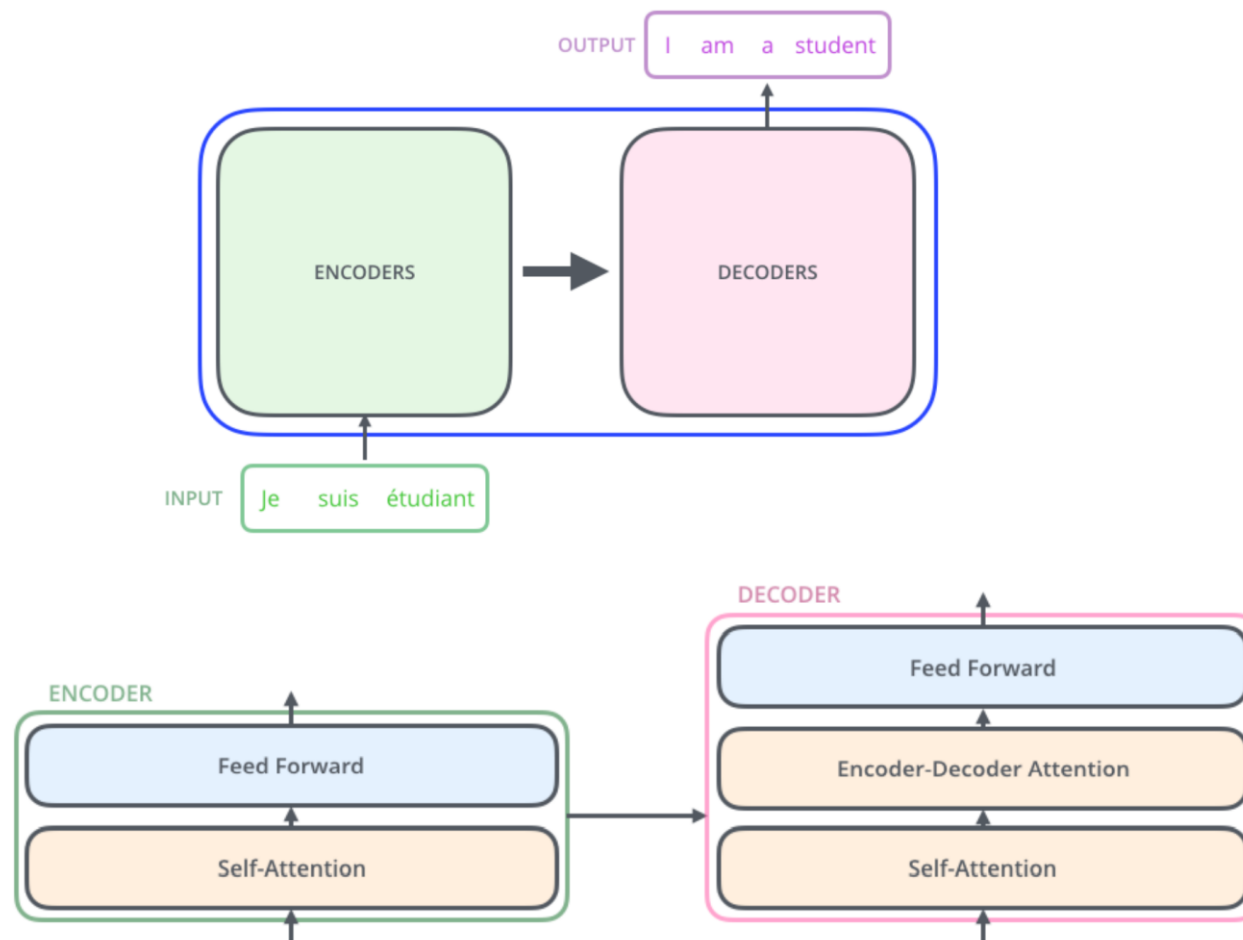




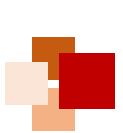
The Transformer



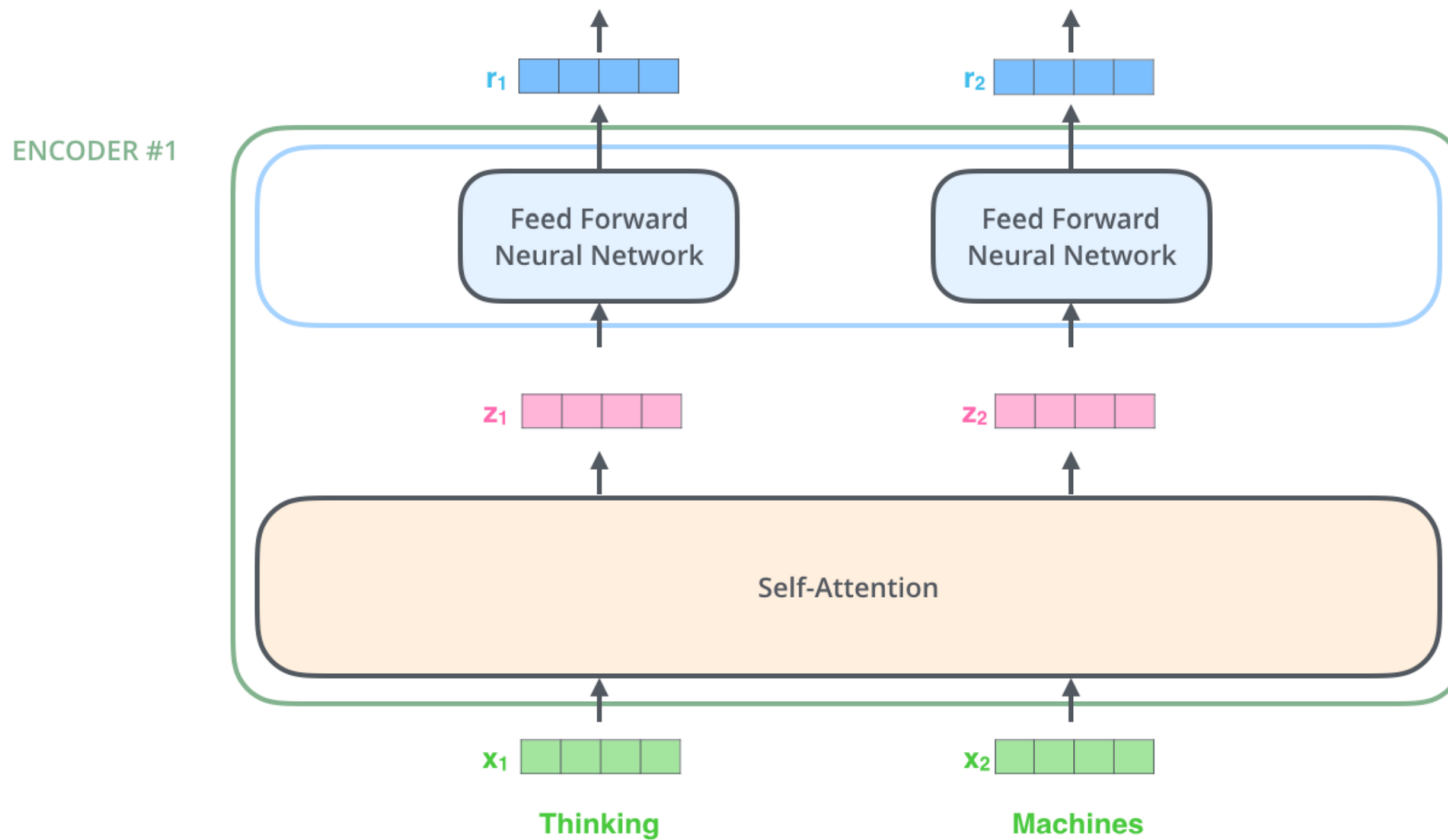
- Encoder-decoder framework



<http://jalammar.github.io/illustrated-transformer/>



Encoder network



<http://jalammar.github.io/illustrated-transformer/>





Self-attention

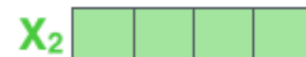
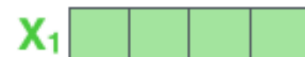


Input

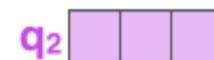
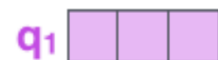
Thinking

Machines

Embedding

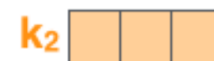
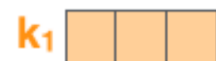


Queries



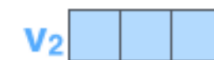
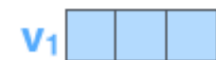
W^Q

Keys

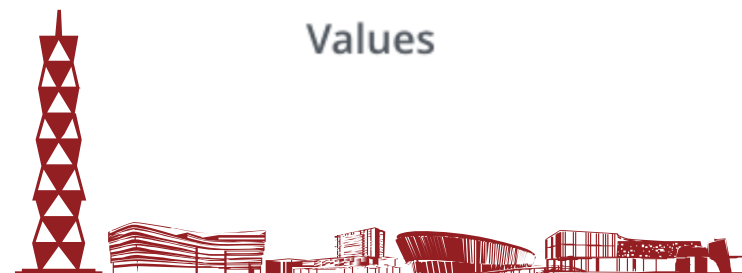


W^K

Values

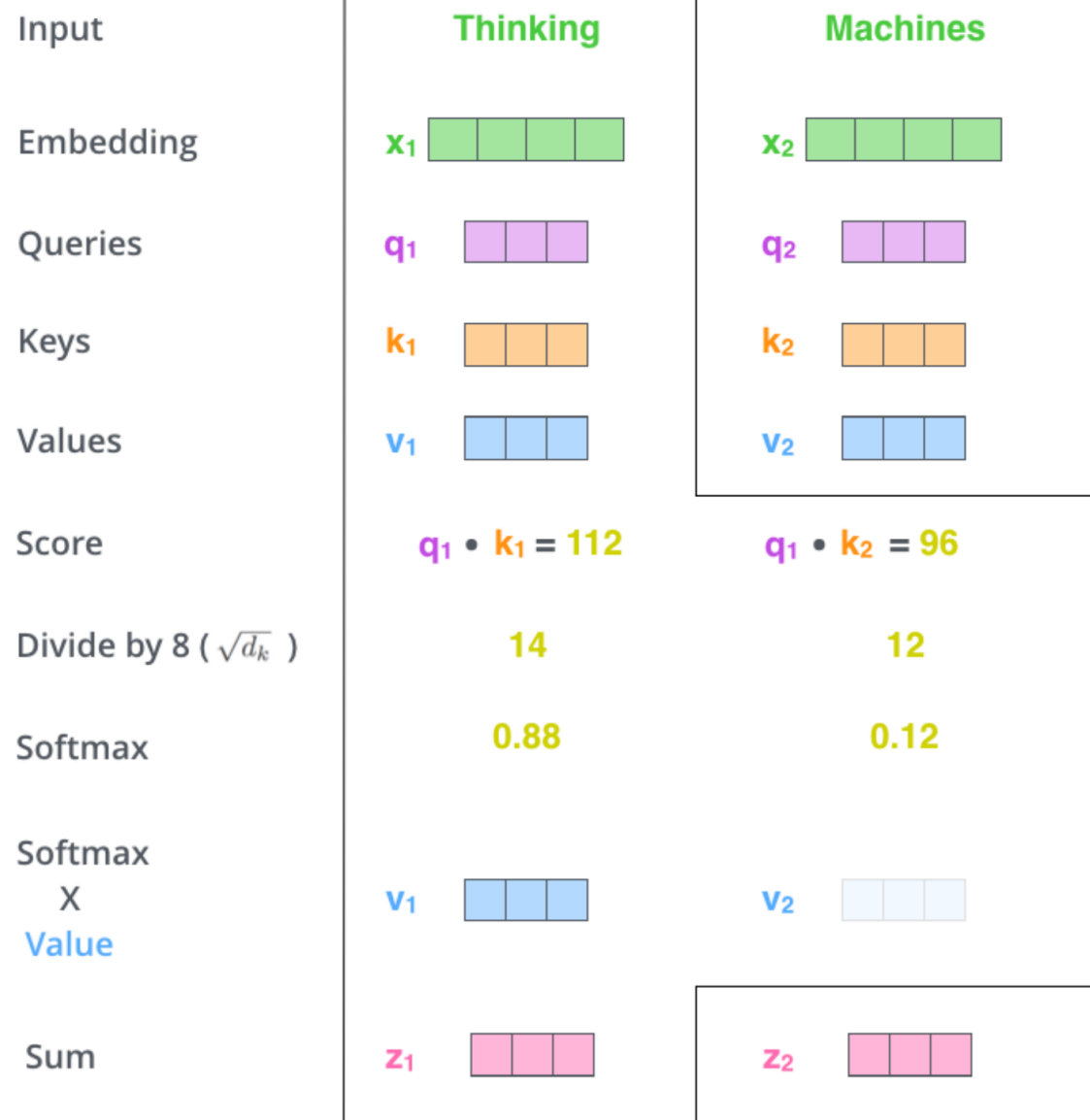


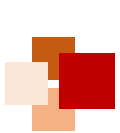
W^V





Self-attention





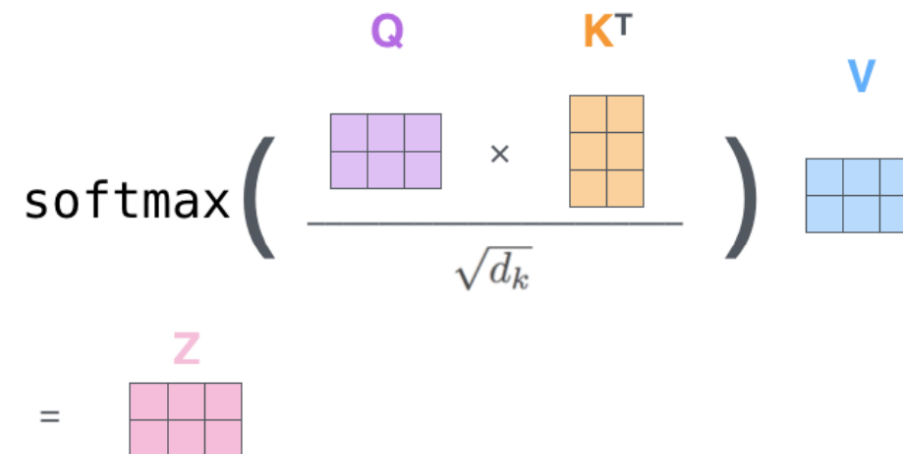
Self-attention in matrix calculation



$$X \times W^Q = Q$$


$$X \times W^K = K$$

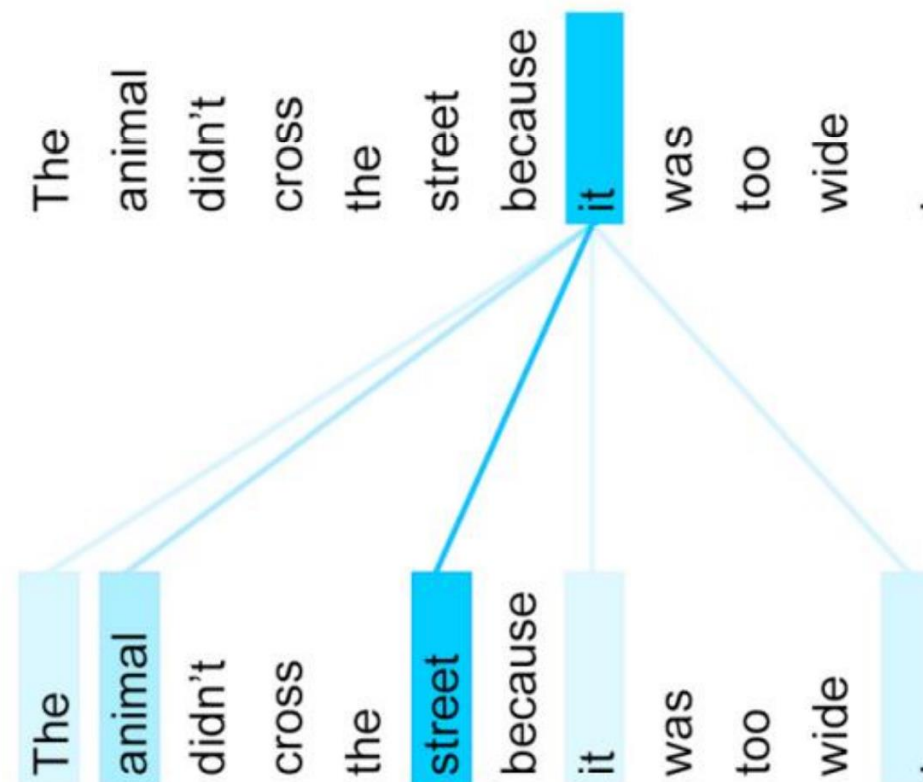
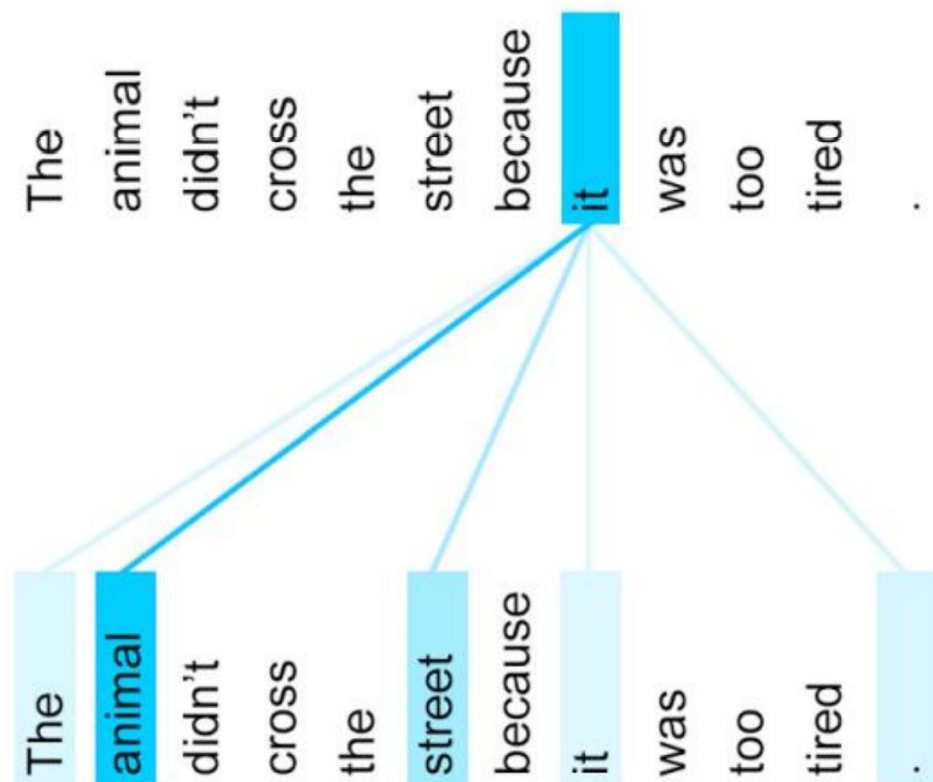

$$X \times W^V = V$$


$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$


Self-attention example



- Self-attention layers learn that “it” could refer to different entities in the different contexts



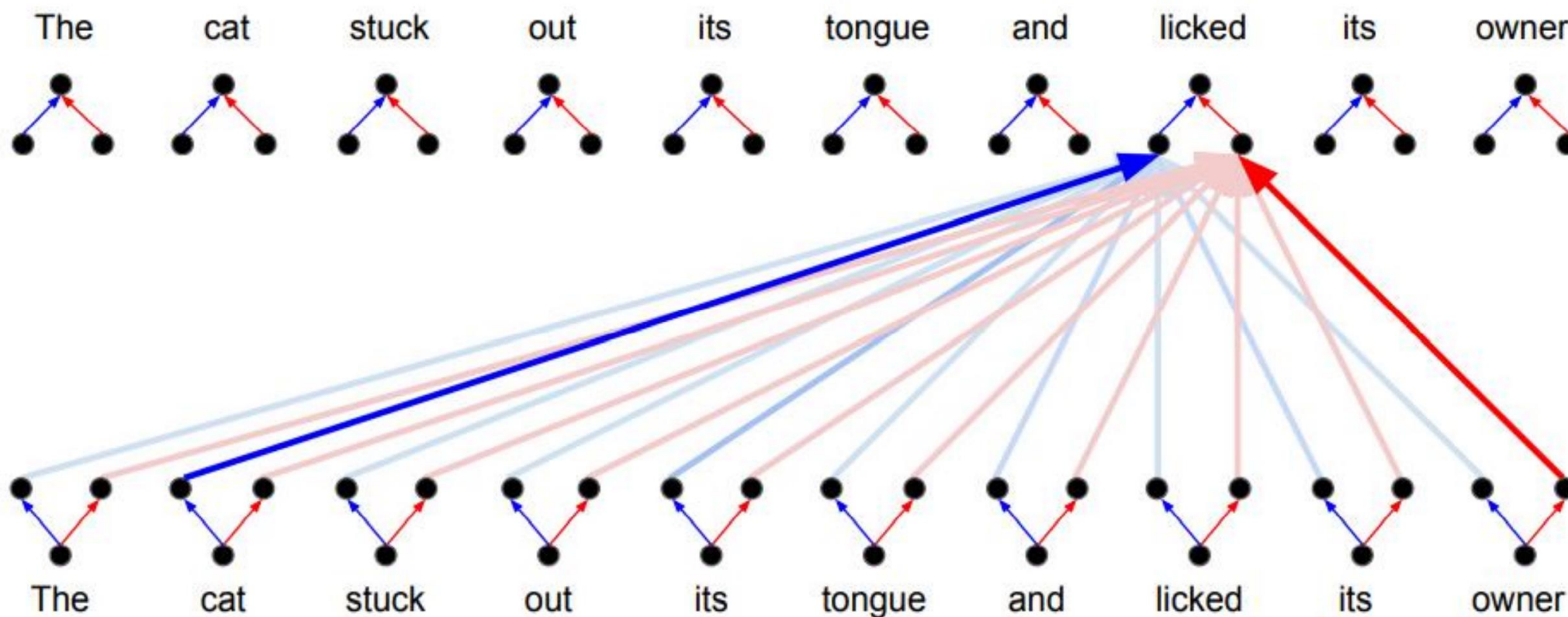
Visualization of the 5th to 6th self-attention layer in the encoder



Multi-head attention



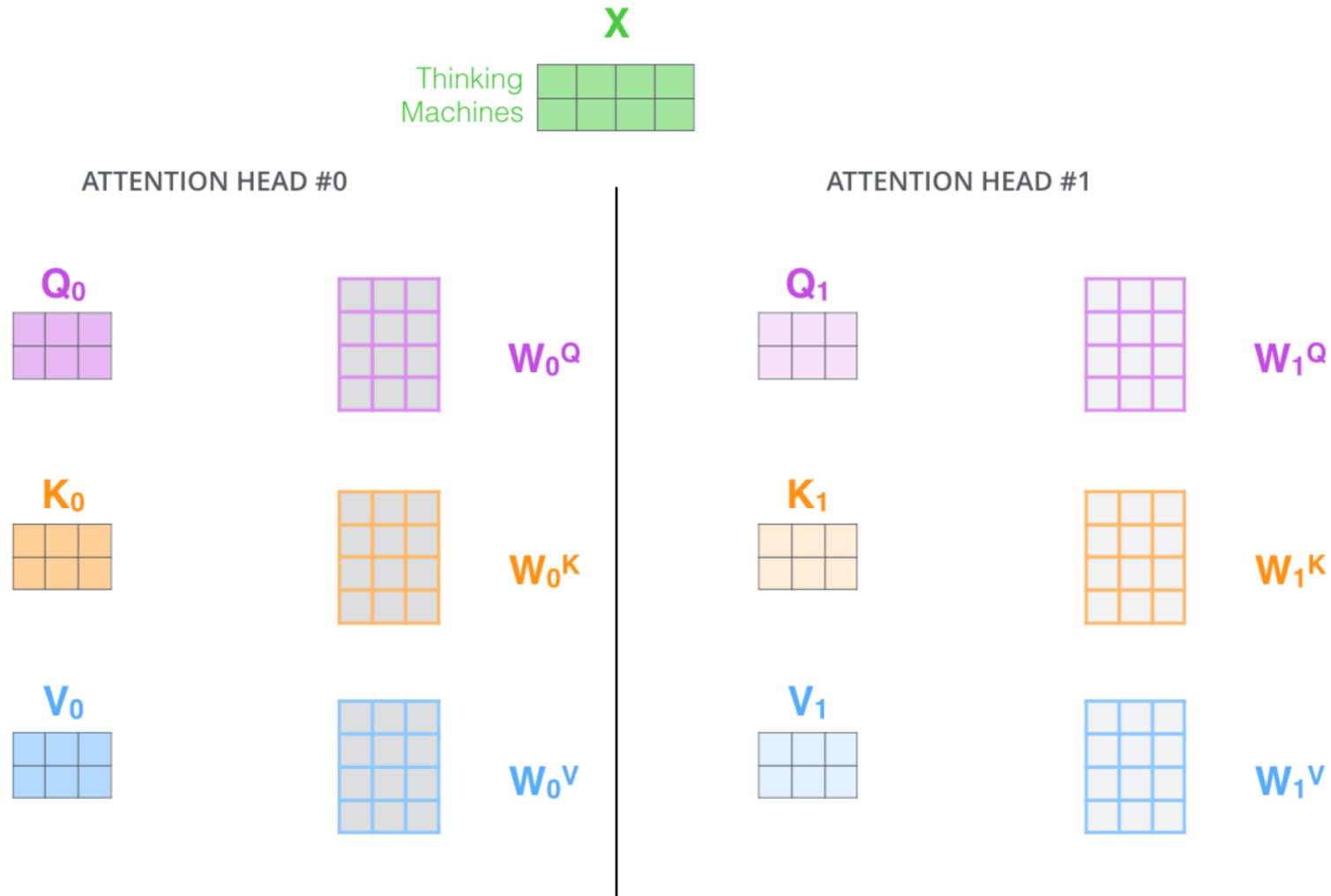
- Parallel attention layers with different linear transformations on input and output



Ashish Vaswani & Anna Huang, Stanford CS224n

Multi-head attention

- Parallel attention layers with different linear transformations on input and output.



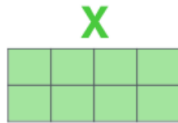
Multi-head attention

- Parallel attention layers with different linear transformations on input and output.

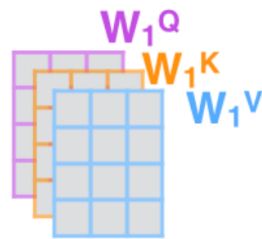
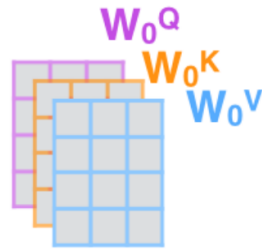
1) This is our input sentence*

Thinking
Machines

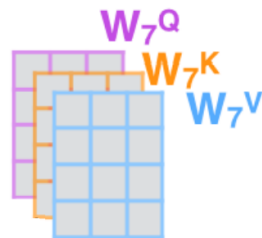
2) We embed each word*



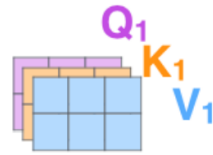
3) Split into 8 heads.
We multiply X or R with weight matrices



...



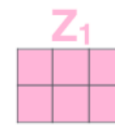
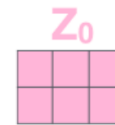
4) Calculate attention using the resulting $Q/K/V$ matrices



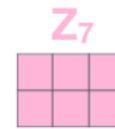
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



...

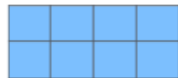


W^O



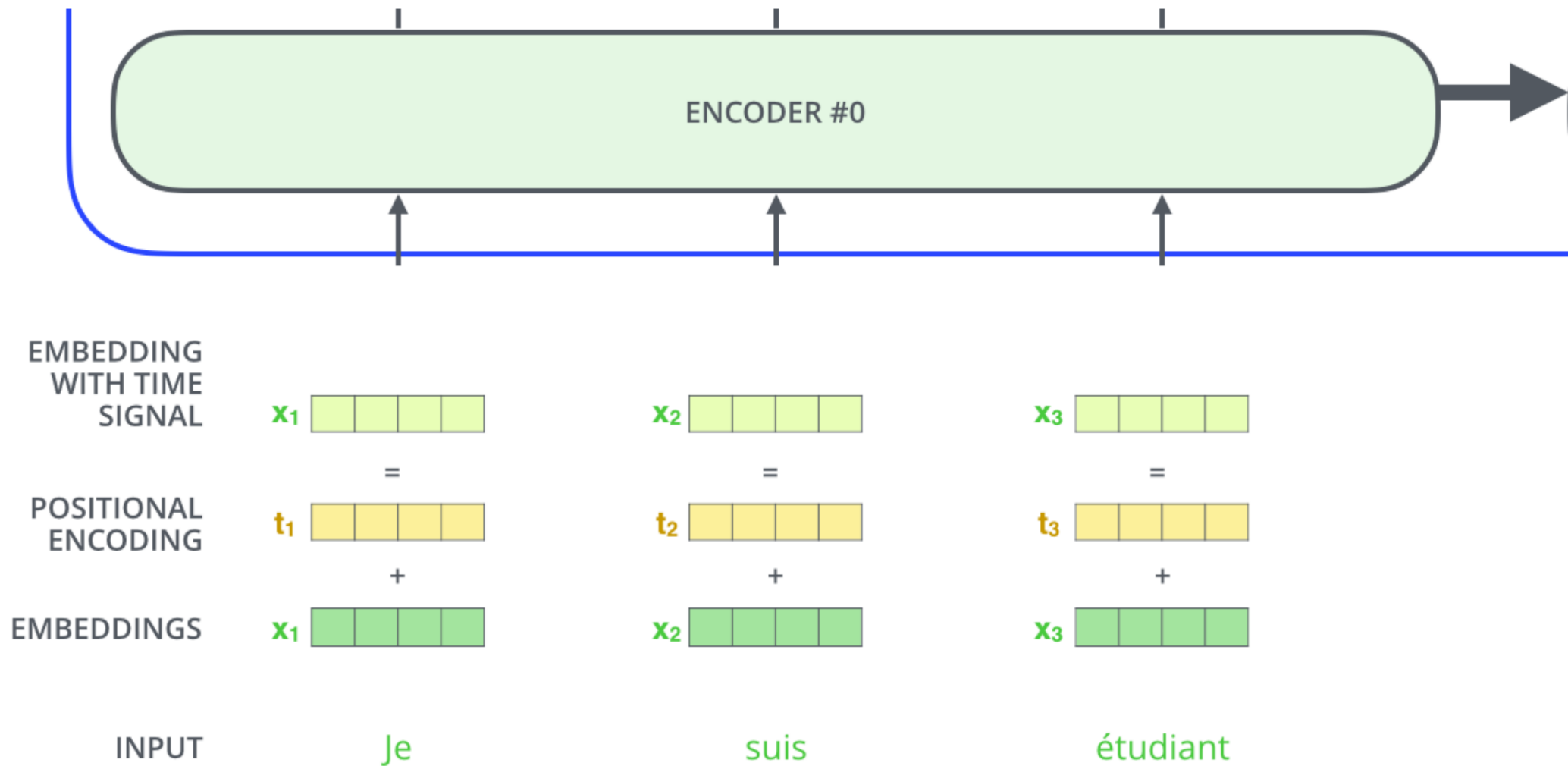
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R



Positional encoding

- Encoder network: the order information



Positional encoding



- Sinusoid
 - Can extrapolate beyond maximum sequence length at test time
 - Represent periodicity of positions: a continuous way of binary encoding of positions

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{emb}}),$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{emb}}),$$

- where i is the index of component in the word encoding
- Learned
 - Rather straightforward
 - Cannot extrapolate



Positional encoding



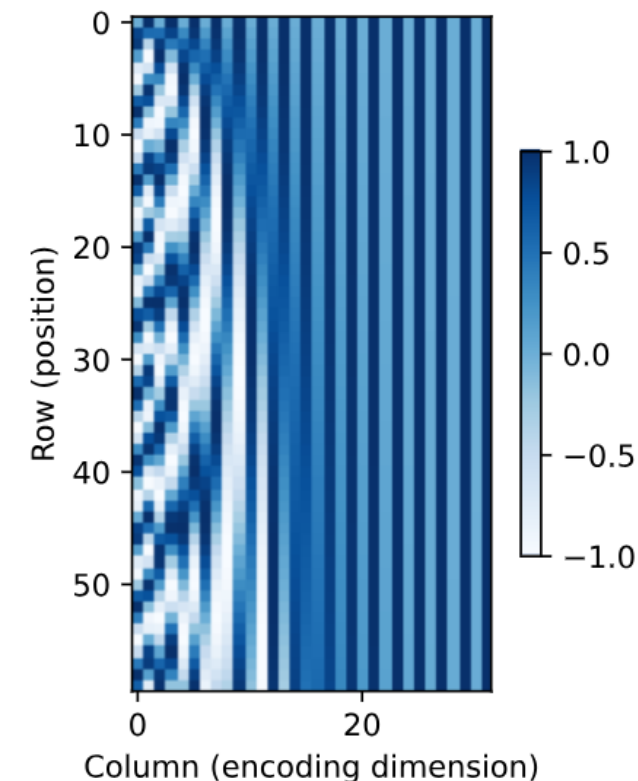
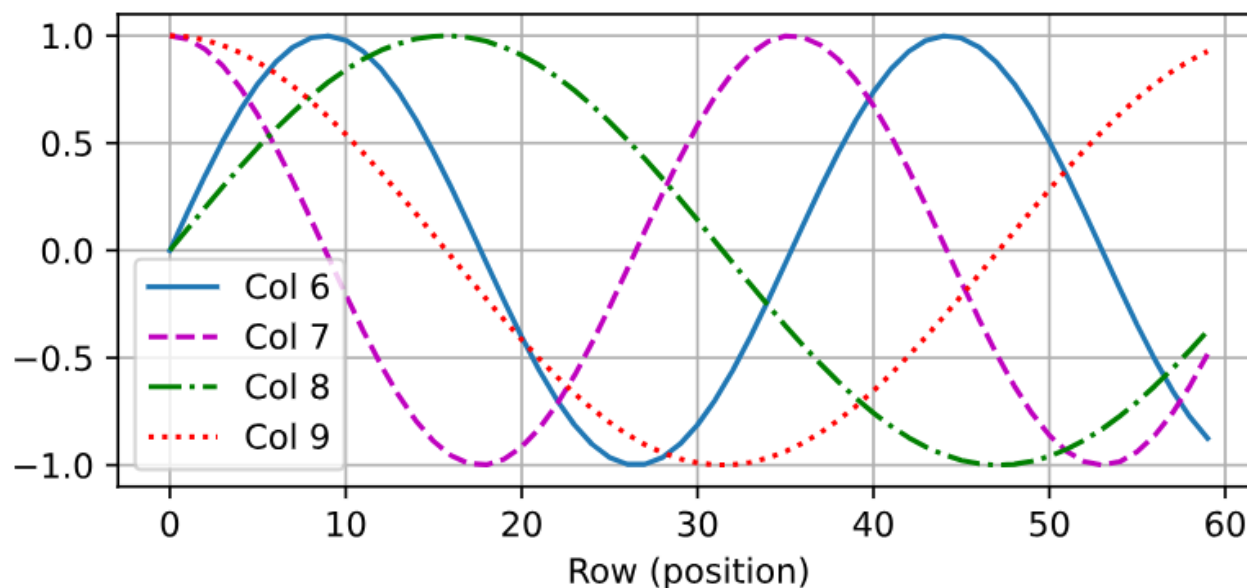
PYTORCH

MXNET

JAX

TENSORFLOW

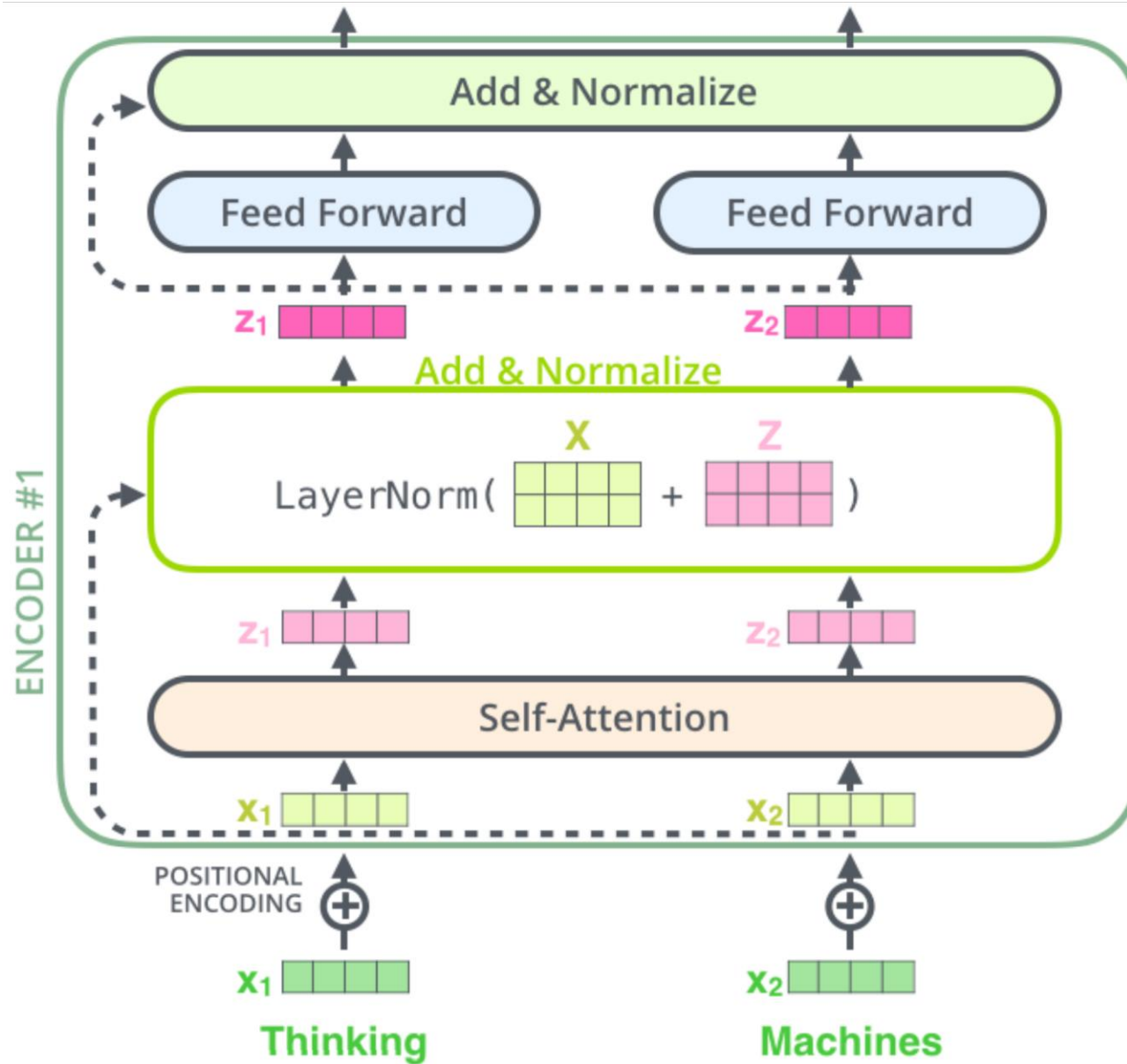
```
encoding_dim, num_steps = 32, 60
pos_encoding = PositionalEncoding(encoding_dim, 0)
X = pos_encoding(torch.zeros((1, num_steps, encoding_dim)))
P = pos_encoding.P[:, :X.shape[1], :]
d2l.plot(torch.arange(num_steps), P[0, :, 6:10].T, xlabel='Row (position)',
         figsize=(6, 2.5), legend=["Col %d" % d for d in torch.arange(6, 10)])
```



Source: Online book “Dive into Deep Learning”, Section 11.6:

https://d2l.ai/chapter_attention-mechanisms-and-transformers/self-attention-and-positional-encoding.html

Residual connections



<http://jalammar.github.io/illustrated-transformer/>

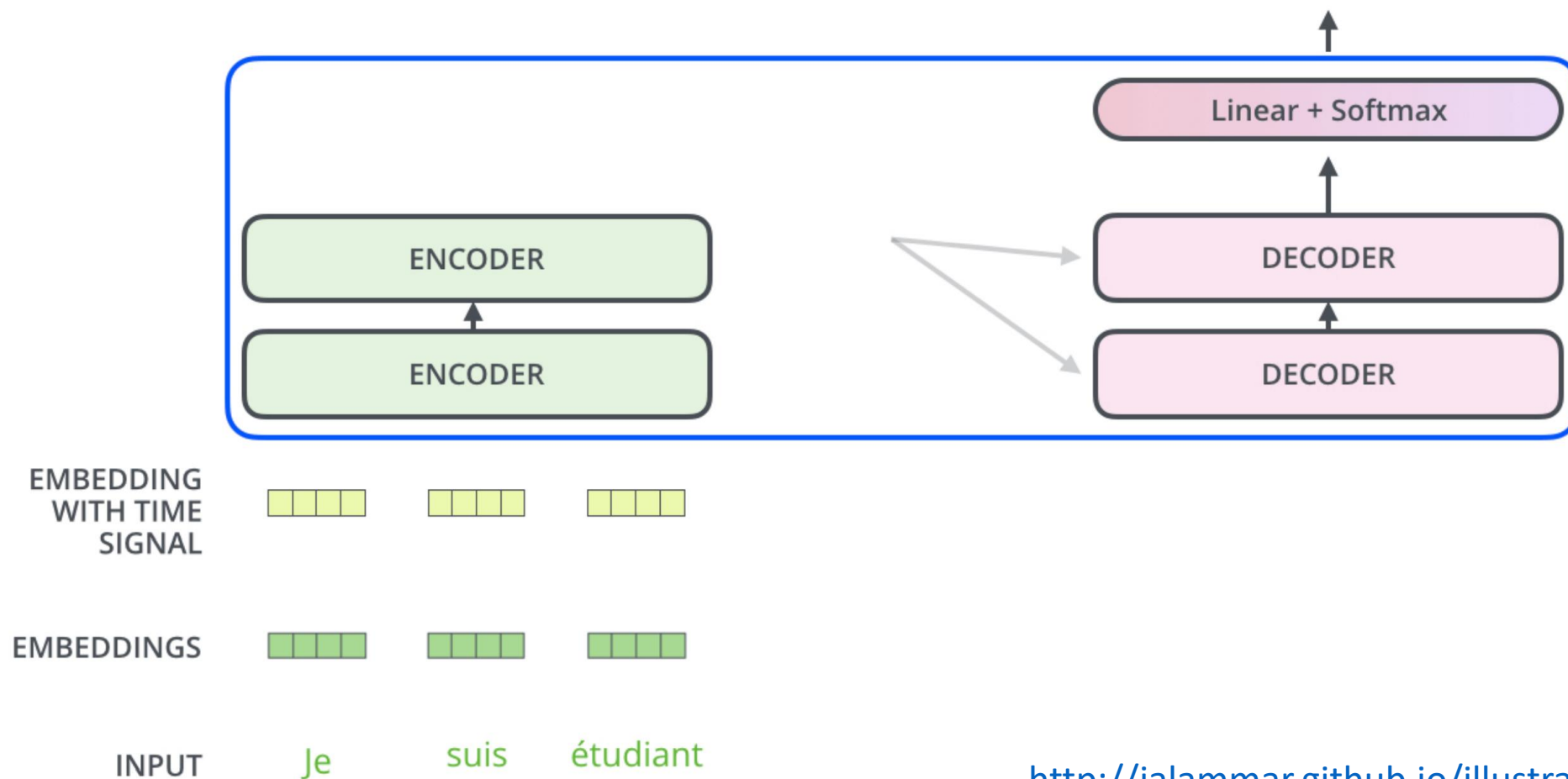


Decoder network



Decoding time step: 1 2 3 4 5 6

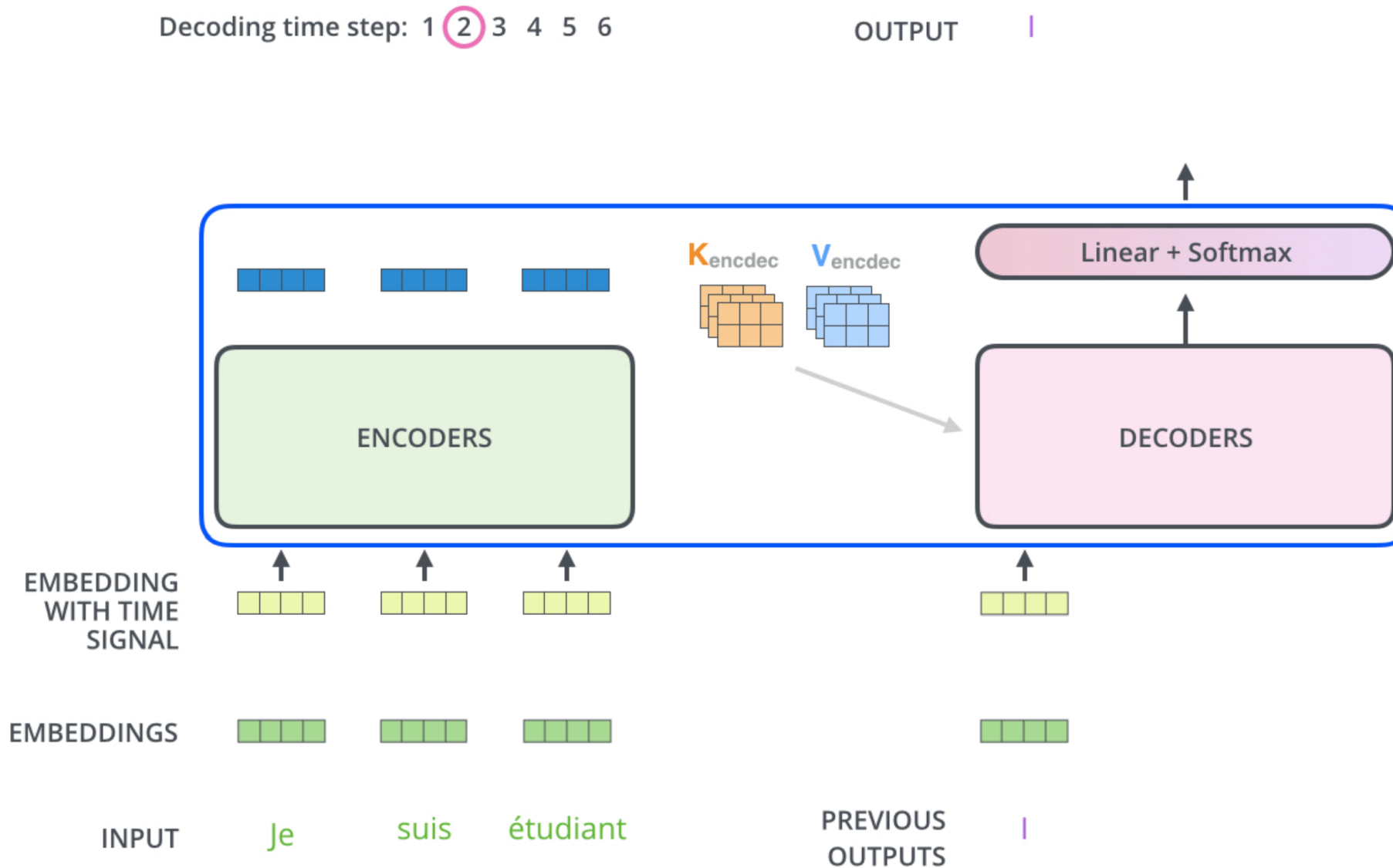
OUTPUT



<http://jalammar.github.io/illustrated-transformer/>



Decoder network

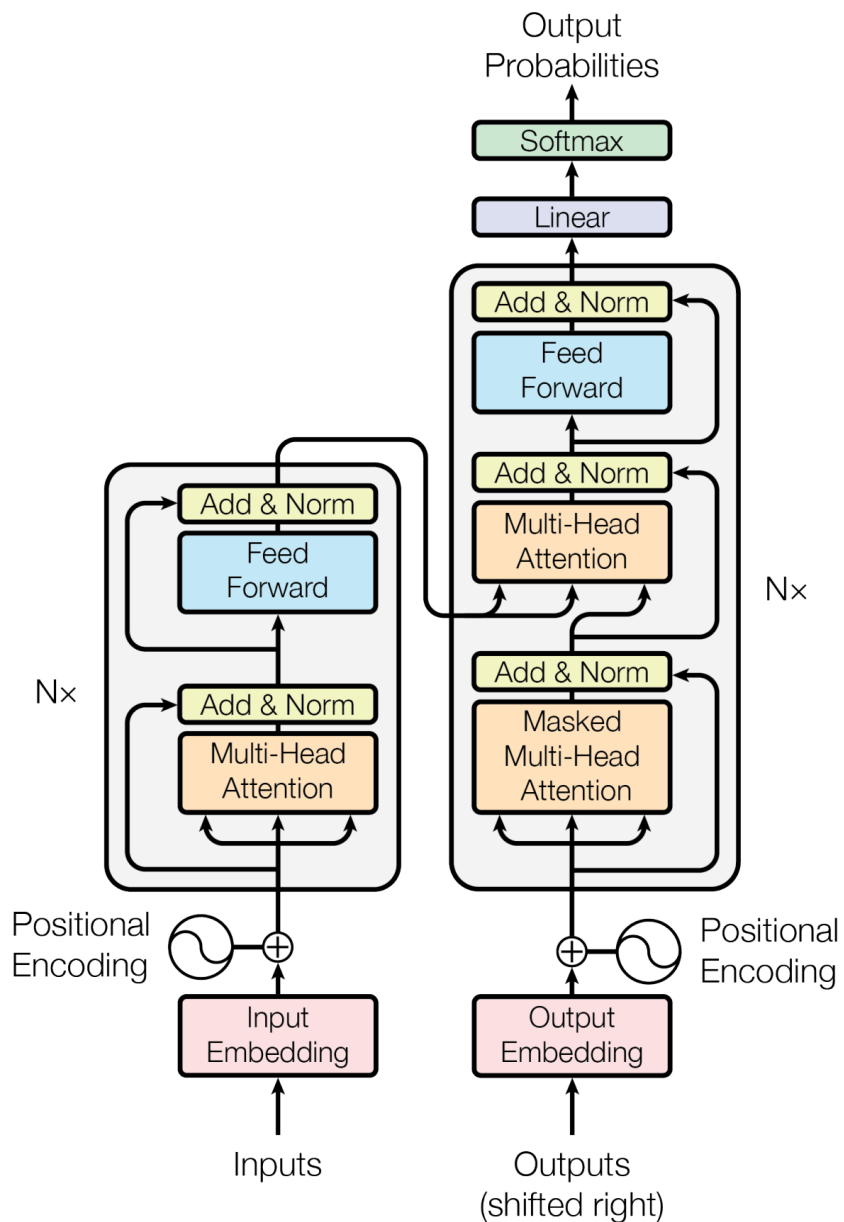


<http://jalammar.github.io/illustrated-transformer/>

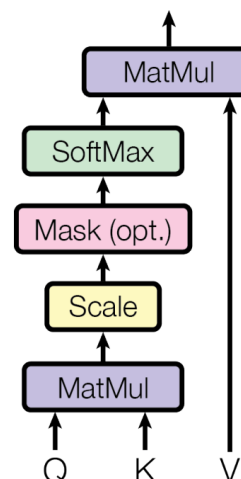




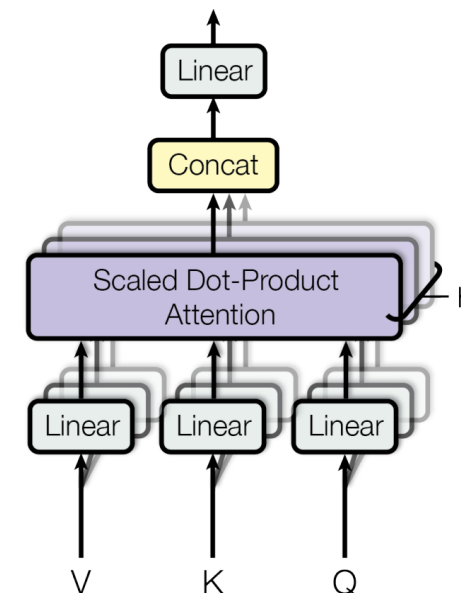
Overall architecture of Transformer



Scaled Dot-Product Attention



Multi-Head Attention



Vaswani et al. "Attention is all you need" , NIPS 2017.



BERT



Differences in pre-training model architectures

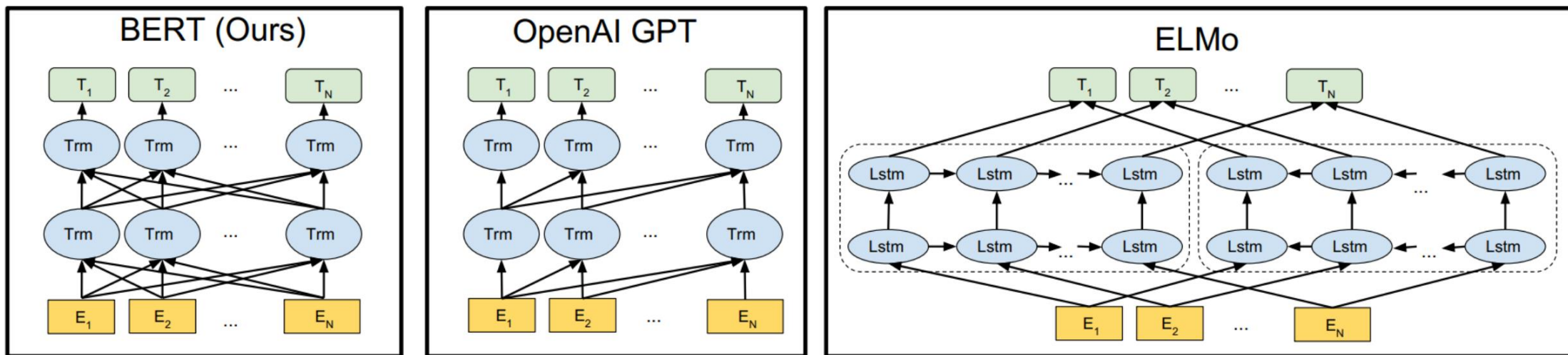


Figure 1: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTM to generate features for downstream tasks. Among three, only BERT representations are jointly conditioned on both left and right context in all layers.

Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, NAACL-HLT 2019.

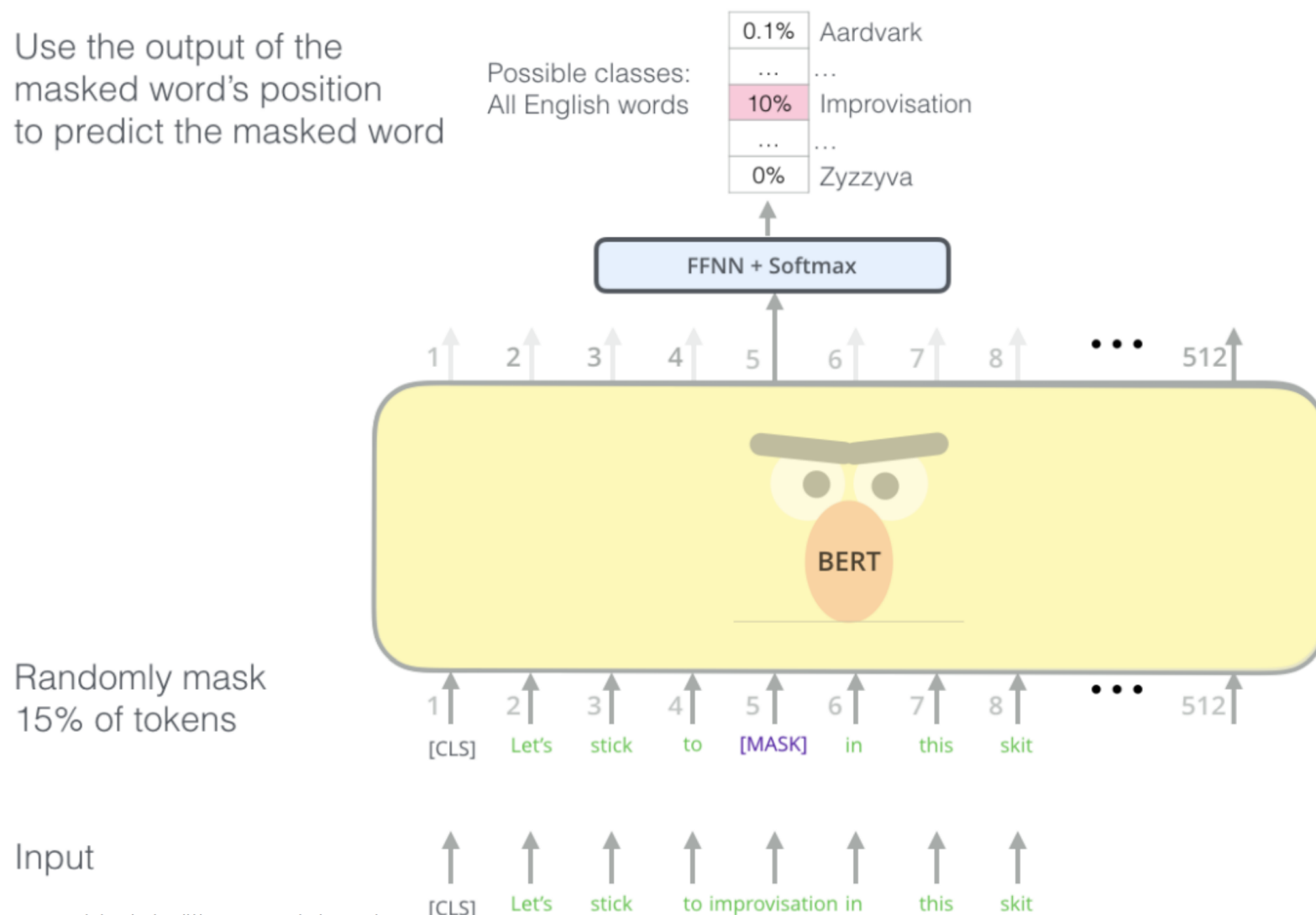
- **ELMo**: Embeddings from Language Models (Matthew Peters et al. 2018)
- **OpenAI GPT**: Generative Pre-trained Transformer (Alec Radford et al. 2018)
- **BERT**: Bidirectional Encoder Representations from Transformers (Jacob Devlin et al. 2019)



Training task 1: Masked Language Model



- For every input sequence, randomly select 15% of tokens (not more than 20 per sequence):
 - For 80% of time: replace the word with the [MASK] token
 - For 10% of time: replace the word with a random word
 - For 10% of time: keep the word unchanged



<http://jalammar.github.io/illustrated-bert/>

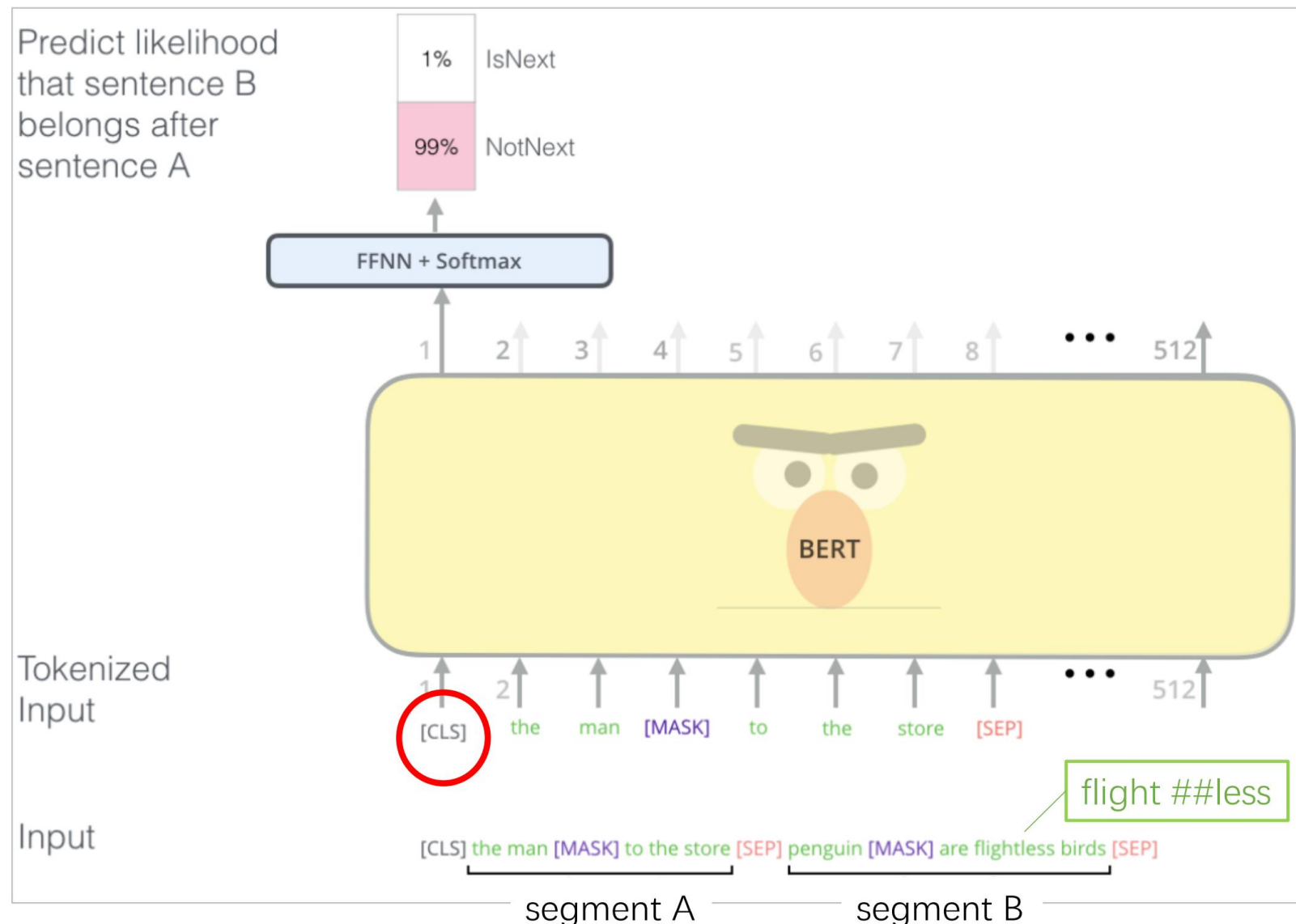




Training task 2: Next Sentence Prediction



- For every input document as a sentence-token 2D list, randomly select a split over sentences:
 - Store segment A
 - For 50% of time, sample random sentence split from another document as segment B
 - For 50% of time, use the actual sentence as segment B

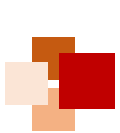


■ Why does BERT work?



- It leverages huge unlabeled and high-quality data
 - 7000 books
 - Wikipedia
 - Together: 3.3 billion words
- Multi-head self-attention blocks in Transformer:
 - Modeling the intra- and extra- word-word relations
 - Parallelable within instance and thus efficient
- Task similarity: masked language modeling + next sentence prediction

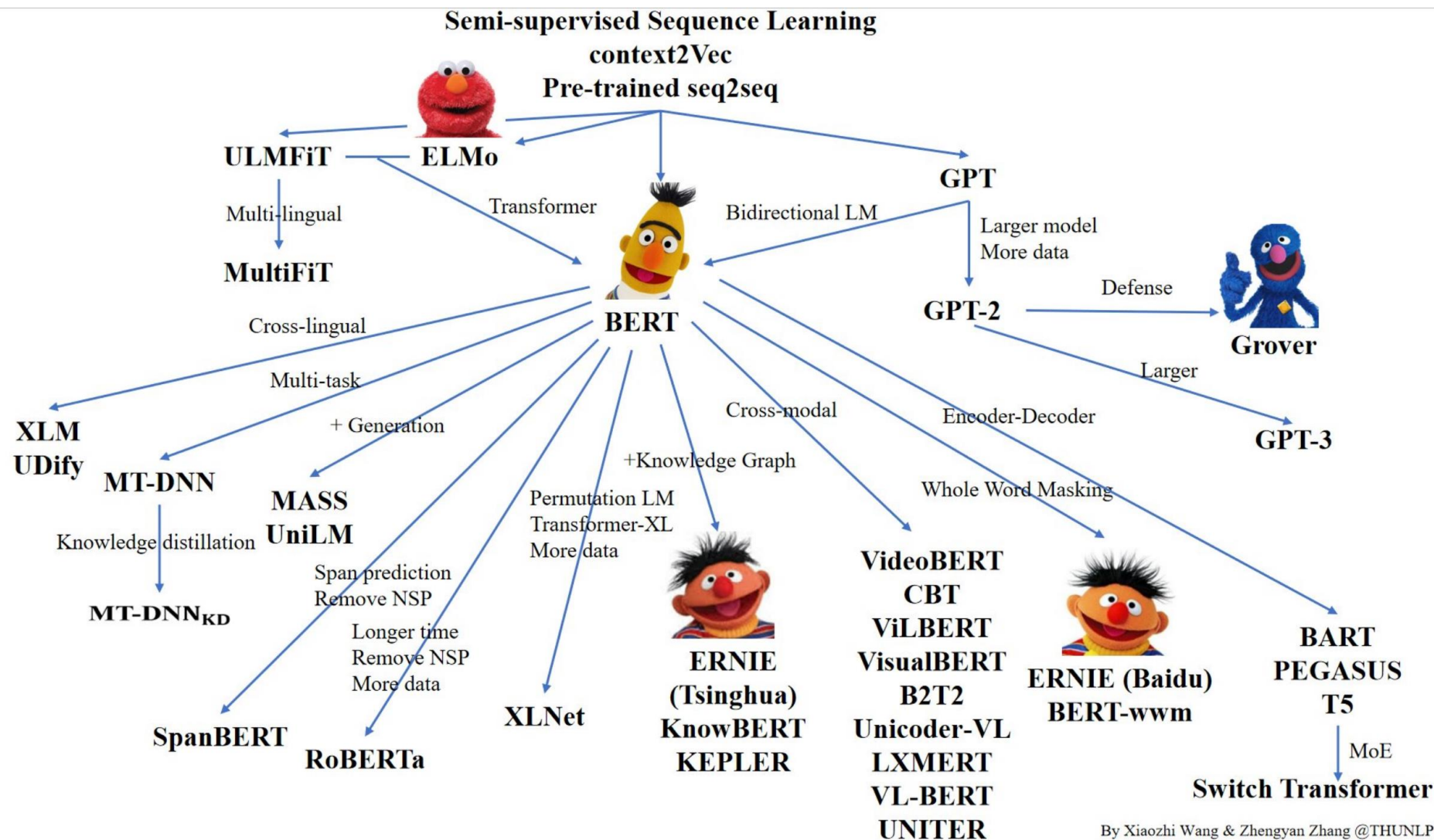




Explosion of pre-trained LMs



上海科技大学
ShanghaiTech University



(This is only by Sep 2019!)



GPT



- ▶ “ELMo with transformers” (works better than ELMo)
- ▶ Train a single unidirectional transformer LM on long contexts

- ▶ GPT2: trained on 40GB of text collected from upvoted links from reddit

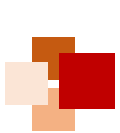
- ▶ 1.5B parameters — by far the largest of these models trained as of March 2019

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

- ▶ Because it's a language model, we can **generate** from it

Radford et al. (2019)

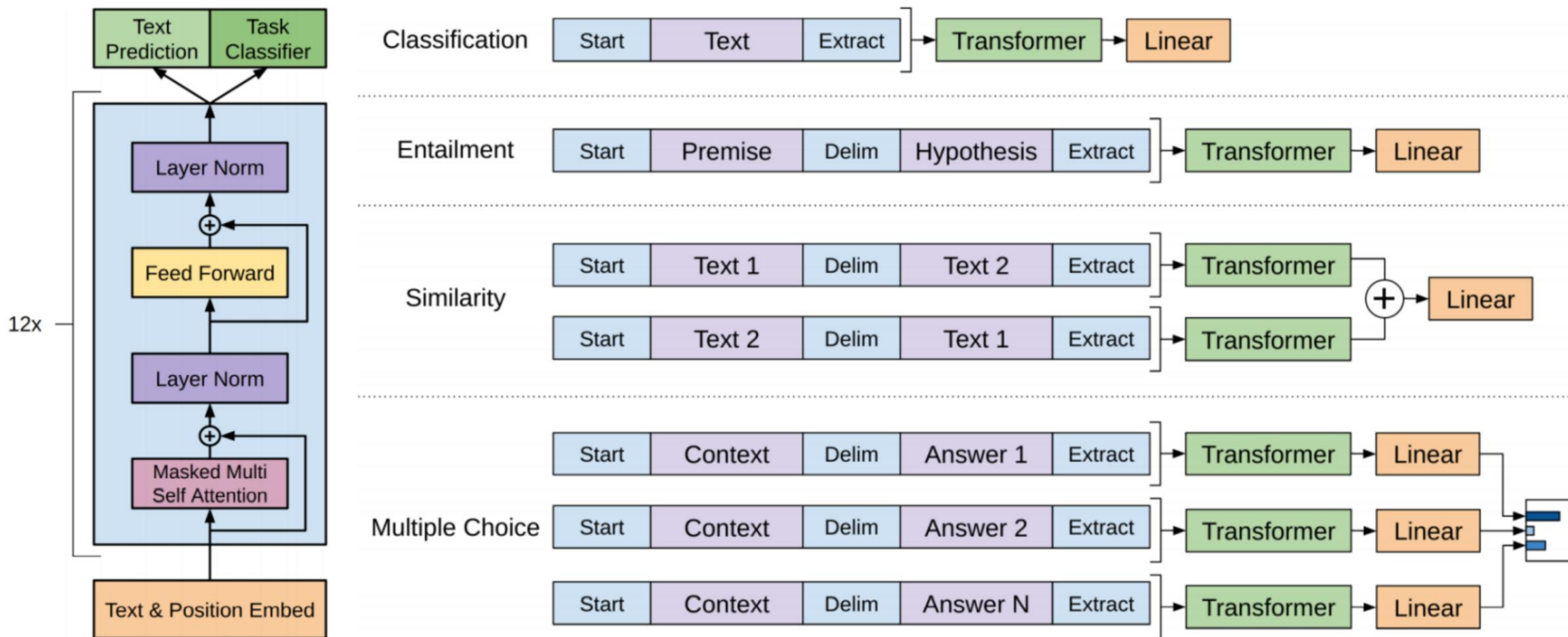




GPT: Transformer language pre-training



- We can pre-train a language model for NLP tasks
 - The pre-trained model is then fine-tuned on target tasks



Radford et al. "Improving language understanding by generative pre-training", 2018.





GPT-2



- A big Transformer LM, with only the decoder side
- Pretrained on a WebText dataset – text from 45 million website links
- Notable for its large size (1.5 billion parameters) on its release
- **Zero-shot learning**: no supervised training data, ask LM to generate from a prompt

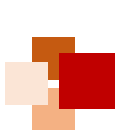
SYSTEM PROMPT (HUMAN-WRITTEN)	<i>Miley Cyrus was caught shoplifting from Abercrombie and Fitch on Hollywood Boulevard today.</i>
MODEL COMPLETION (MACHINE-WRITTEN, SECOND TRY)	The 19-year-old singer was caught on camera being escorted out of the store by security guards. The singer was wearing a black hoodie with the label 'Blurred Lines' on the front and 'Fashion Police' on the back. Scroll down for video Shoplifting: Miley Cyrus was caught shoplifting from Abercrombie and Fitch on Hollywood Boulevard today (pictured) The singer was wearing a black hoodie with the label 'Blurred Lines' on the front and 'Fashion Police' on the back The singer was also wearing a pair of black-rimmed glasses, a black jacket, black jeans and black sandals. She was carrying a pair of black and white striped gloves and a small black bag.



Image credit: Jay Alammar

slide credit:
OpenAI

Radford et al. "Language models are unsupervised multitask learners", 2018.



- ▶ GPT-2 but even larger: 1.3B -> 175B parameter models

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

- ▶ Trained on 570GB of Common Crawl
- ▶ 175B parameter model’s parameters alone take >400GB to store (4 bytes per param). Trained in parallel on a “high bandwidth cluster provided by Microsoft”

Brown et al. (2020)



Finetuning approach



- This is the “normal way” of doing learning in models like GPT-2

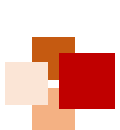
Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Brown et al. (2020)





GPT-3: few-shot learning



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← examples
3	peppermint => menthe poivrée	←
4	plush girafe => girafe peluche	←
5	cheese =>	← prompt

Examples: https://docs.google.com/document/d/1dIpx4lbT9SQH-ONwf8kwBAyq6BuFcGO6c_4RjZif8zE/edit

Brown et al. (2020)





Summary



- In this lecture we have learned
 - The basics of Natural Language Processing (NLP)
 - The background and main ideas of attention mechanisms
 - Transformer uses attention mechanisms only, without using RNN
 - To capture relationships among words and computational efficiency, Transformer uses self-attention, multi-head attention, and positional encodings
 - Based on Transformer, a few pre-training language models have been proposed to achieve great success, e.g. BERT and GPT series





References



- Chapter 16 of Aurélien Géron' s book "Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow" (Ed. 2) , O' Reilly, 2019.
- Chapters 23 and 24 of Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach" (Ed. 4), Pearson, 2021.
- Bahdanau et al. "Neural machine translation by jointly learning to align and translate" , ICLR 2015.
- Luong et al. "Effective approaches to attention-based neural machine translation" , EMNLP 2015.
- Vaswani et al. "Attention is all you need" , NIPS 2017. (**Transformer**)
- Radford et al. "Improving language understanding by generative pre-training" , 2018. (**GPT-1**)
- Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" , NAACL-HLT 2019. (**BERT**)
- Radford et al. "Language models are unsupervised multitask learners" , 2018. (**GPT-2**)
- Brown et al. "Language models are few-shot learners" , NIPS 2020 (**GPT-3**)

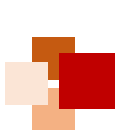




Appendix: Overview of NLP

PPT adapted from lecture notes of Prof. Kewei Tu, CS274 “NLP”, ShanghaiTech (Spring 2022)
and of Prof. Hongning Wang, CS6501 “Text Mining”, University of Virginia (2019)





What is NLP?



- Automating the **analysis**, **generation** and **acquisition** of human (“natural”) languages

Arabic text كلب هو مطاردة صبي في الملعب.

How can a computer make **sense** out of this **string**?

- | | |
|-------------------|---|
| Morphology | - What are the basic units of meaning (words)?
- What is the meaning of each word? |
| Syntax | - How are words related with each other? |
| Semantics | - What is the “combined meaning” of words? |
| Pragmatics | - What is the “meta-meaning”? (speech act) |
| Discourse | - Handling a large chunk of text |
| Inference | - Making sense of everything |



A brief history of NLP



- Early enthusiasm (1950' s): machine translation
 - Too ambitious
 - Bar-Hillel report (1960) concluded that fully-automatic high-quality translation could not be accomplished without knowledge (Dictionary + Encyclopedia)
- Less ambitious applications (late 1960' s to early 1970' s): Limited success, failed to scale up
 - Speech recognition
 - Dialogue (Eliza)
 - Inference and domain knowledge
- Real world evaluation (late 1970' s – now)
 - Story understanding (late 1970' s & early 1980' s)
 - Large-scale evaluation of speech recognition, text retrieval, information extraction (1980 – now)
 - Statistical approaches enjoy more success (first in speech recognition & retrieval, later others)
- Current trend:
 - Boundary between statistical and symbolic approaches is disappearing
 - We need to use all the available knowledge
 - Application-driven NLP research (Bioinformatics, Web, Question answering...)
 - Deep learning for NLP





Tasks of NLP



- **Tokenization/Segmentation:** Split text into words and sentences
- **Part-of-Speech (POS) tagging:** Marking up a word in a text (corpus) as corresponding to a particular part of speech
- **Named entity recognition (NER):** Determine text mapping to proper names
- **Syntactic parsing:** Grammatical analysis of a given sentence, conforming to the rules of a formal grammar
- **Relation extraction:** Identify the relationships among named entities
- **Logic inference:** Convert chunks of text into more formal representations





Probabilistic language modeling



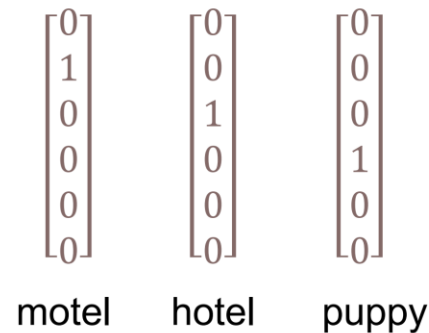
- **Goal:** To compute the probability of a sentence (sequence of words)
 $P(w_1, w_2, w_3, \dots, w_n)$.
- **Applications:**
 - Machine translation, spell correction, speech recognition, etc.
- **A trivial language model (LM):** estimating sentence probability from its frequency in a corpus
 - **Issue:** It's impossible for a corpus to cover all possible sentences
- **Methods:**
 - **n-gram:** Probability of each word is conditioned on the preceding $n - 1$ words
 - **Recurrent neural networks:** Probability of each word is conditioned on a hidden vector summarizing all the preceding words
 - **Transformer:** Probability of each word is computed by attending to preceding words
 - Other methods: e.g. grammar-based



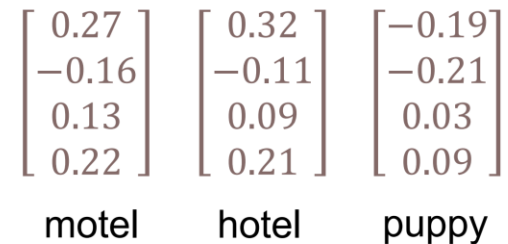
Word representation

- In traditional NLP, we regard words as discrete symbols
 - Issue: no natural notion of similarity
- **Word vectors** (also known as **word embeddings**): Representing each word in a vector space, with the following advantages:
 - Can capture similarity between words with vector distances
 - Easier to process in many ML systems
 - Can be learned from data

one-hot vectors



dense vectors



project to 2D

